

数据结构

Morning Flower and Evening Oath

XCPC vector 模板分册

目录

使用约定	iii
第一章 哈希表	1
1.1 GNU gp_hash_table 与 cc_hash_table	1
第二章 并查集	4
2.1 并查集	4
2.1.1 并查集	4
2.1.2 可撤销并查集	6
第三章 堆	8
3.1 配对堆	8
3.1.1 PBDS 可修改、可合并优先队列	8
3.2 左偏树	9
3.2.1 左偏树可并小根堆	9
第四章 树状数组	11
4.1 树状数组与第 k 小	11
第五章 线段树	12
5.1 线段树基础	12
5.1.1 通用懒标记线段树与边界搜索	12
5.1.2 通用线段树与左右边界搜索	17
5.1.3 区间加与区间和	22
5.1.4 线段树：区间乘、加、赋值与模意义求和	23
5.1.5 线段树维护单调栈：区间加与接雨水	26
5.2 李超线段树	31
5.2.1 精确线段李超树：最大值编号	31
5.2.2 离散李超树与最优直线编号	37
第六章 二叉搜索树 & 平衡树	40
6.1 二叉搜索树 & 平衡树	40
6.1.1 GNU rope 与版本共享	40
6.1.2 PBDS 有序树与顺序统计	42
6.2 Treap	44
6.2.1 FHQ Treap：可重集合与顺序统计	44
6.2.2 隐式 Treap：顺序反转、位翻转、加值与区间和	48
6.2.3 状态序列 Treap：分组区间 GCD	53
6.3 Splay 树	56
6.3.1 Splay：可重集合与顺序统计	56
6.4 替罪羊树	61
6.4.1 替罪羊树：可重集合与顺序统计	61

第七章 可持久化数据结构	65
7.1 可持久化线段树	65
7.1.1 主席树：静态区间第 k 小	65
7.1.2 可持久化数组：历史版本分叉与单点修改	67
7.1.3 可持久化区间加与版本拼接	69
7.1.4 树上路径点权第 k 小	72
7.1.5 静态区间不同数计数（前驱位置主席树）	75
第八章 树套树	78
8.1 树状数组套权值线段树	78
8.1.1 动态区间第 k 小（树状数组套权值线段树）	78
第九章 K-D Tree	82
9.1 动态二维 K-D Tree：单点累加与矩形和	82
第十章 动态树	86
10.1 Link Cut Tree	86
10.1.1 Link-cut tree：动态森林与路径异或	86
10.1.2 动态换根路径乘积总和	91
第十一章 配套知识与验证范围	95
11.1 可持久化区间拼接的懒标记	95
11.2 杭州 2023 K：跨版本拼接应用	96
11.3 并查集模板题的接口范围	97
11.4 动态树路径乘积：虚子树与双向信息	97
11.5 隐式 Treap 位翻转与顺序反转	97
11.6 替罪羊树：重复计数与墓碑清理	97
11.7 线段树维护单调栈：双向扫描与接雨水	98
11.8 动态 K-D Tree：矩形和与重建边界	98
第一部分 跨类依赖	99
附录 A 数论	100
A.1 模算术简介	100
A.1.1 静态模整数	100

使用约定

C++20，默认包含 `bits/stdc++.h` 与 `cassert`，并使用 `std` 命名空间。下标、区间及数值范围以各条目说明为准。DFS 保持递归，需满足题目栈空间条件。

本册主目录沿用 OI Wiki 官方导航的大、中、小层级，各页下列本库实现。PBDS 与 GNU rope 按队内约定纳入平衡树。组合条目与相关挂靠仍见仓库分类审计。跨类引用的已收录组件附在依赖部分；源头文件中的依赖仍须一并检查。

本册尚非全量完成稿；传统版评测仅为历史档案，当前只维护 `vector` 代码。

第一章 哈希表

1.1 GNU gp_hash_table 与 cc_hash_table

GNU libstdc++ 扩展, 包含 ext/pb_ds/assoc_container.hpp。gp_map<K,V> 为探测哈希表, cc_map<K,V> 为碰撞链接哈希表; K 为不超过 64 位的整数, 复用下方 hash64。两者支持 operator[]、find、insert、erase(key)、clear、size、empty; insert 不覆盖已有键, operator[] 对不存在的键插入默认值。只查询时用 find 判断 end。通常查改摊还 O(1), 最坏 O(n), 空间 O(n), 不支持排名或有序前驱。纯键值查改可先试 gp_map; 需要在插入/删除其他键后保留找到的元素引用时可考虑 cc_map, 其节点分配和链接有成本, 速度仍须按题实测。gp_map 修改后重新 find; cc_map 的点迭代器不等于可递增的遍历迭代器, 删除该元素、clear 或销毁后不能保留引用。默认策略不直接调用 resize, 也不照搬 unordered_map::reserve。hash64 使用进程内固定随机种子, 无符号回绕; 只针对整数键, 不保证已知种子下的最坏碰撞。历史 associative_array AC 仅覆盖 gp_map, cc_map 另行验证。

需要排名与有序前驱时, PBDS 平衡树见第 42 页。

```
1 struct hash64
2 {
3     static uint64_t mix(uint64_t x)
4     {
5         x += 0x9e3779b97f4a7c15ULL;
6         x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
7         x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
8         return x ^ (x >> 31);
9     }
10
11     size_t operator()(uint64_t x) const
12     {
13         static const uint64_t seed =
14             chrono::steady_clock::now().time_since_epoch().count();
15         return mix(x + seed);
16     }
17 };
18
19 template <class K, class V> using gp_map = __gnu_pbds::gp_hash_table<K, V, hash64>;
20 template <class K, class V> using cc_map = __gnu_pbds::cc_hash_table<K, V, hash64>;
```

模板题使用：Library Checker associative_array

题意：维护整数键值映射，修改为覆盖赋值，查询不存在的键输出 0。gp_map 用 operator[] 写入，用 find 查询，避免查询插入新键。所需条目中包含 hash64。

所需模板：gp_map。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/associative_array

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int q;
6     cin >> q;
7     gp_map<uint64_t, uint64_t> a;
8     while (q--)
9     {
10         int op;
11         uint64_t k, v;
12         cin >> op >> k;
13         if (op == 0)
14         {
15             cin >> v;
16             a[k] = v;
17         }
18         else
19         {
20             auto it = a.find(k);
21             cout << (it == a.end() ? 0 : it->second) << '\n';
22         }
23     }
24 }
```

模板题使用：Library Checker associative_array

题意：同一关联数组题的 `cc_map` 用法：覆盖赋值，查询不存在的键输出 0。`cc_map` 与 `hash64` 均在 `gp_map` 条目中；本例独立执行验证，不借用 `gp_map` 的运行结果。

所需模板：`gp_map`。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：https://judge.yosupo.jp/problem/associative_array

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int q;
6     cin >> q;
7     cc_map<uint64_t, uint64_t> a;
8     while (q--)
9     {
10         int op;
11         uint64_t k, v;
12         cin >> op >> k;
13         if (op == 0)
14         {
15             cin >> v;
16             a[k] = v;
17         }
18         else
19         {
20             auto it = a.find(k);
21             cout << (it == a.end() ? 0 : it->second) << '\n';
22         }
23     }
24 }
```

第二章 并查集

2.1 并查集

2.1.1 并查集

dsu(n), 点号 $0..n-1$, $n \geq 0$ 。find(x) 路径减半; same(x,y) 判断同集合; merge(x,y) 把 y 的根接到 x 的根, 已同集合返回 false。size(x) 返回所在集合大小。groups() 返回所有非空分组, 组内点号递增、组间按当前根编号排列; 可能压缩路径, 但不改变分组。fa、sz 可读, sz 仅根处有效; 不要外部改写。本版只做路径减半, 不按大小或秩合并; 单次 find 最坏 $O(n)$ 。空间 $O(n)$, groups 另需 $O(n)$ 输出与工作空间, 并执行 n 次 find。空结构可调用 groups, 不能查询顶点。

```
1 struct dsu
2 {
3     vector<int> fa, sz;
4
5     dsu(int n) : fa(n), sz(n, 1) { iota(fa.begin(), fa.end(), 0); }
6
7     int find(int x)
8     {
9         while (x != fa[x]) x = fa[x] = fa[fa[x]];
10        return x;
11    }
12
13    bool same(int x, int y) { return find(x) == find(y); }
14
15    bool merge(int x, int y)
16    {
17        x = find(x), y = find(y);
18        if (x == y) return false;
19        sz[x] += sz[y], fa[y] = x;
20        return true;
21    }
22
23    int size(int x) { return sz[find(x)]; }
24
25    vector<vector<int>> groups()
26    {
27        vector<vector<int>> res(fa.size());
28        for (int i = 0; i < int(fa.size()); i++) res[find(i)].push_back(i);
29        res.erase(
30            remove_if(res.begin(), res.end(), [&](const auto &v) { return v.empty(); }),
31            res.end());
32        return res;
33    }
34};
```

模板题使用：Luogu P3367

题意：维护集合合并与两点是否同集合。题目点号 $1..n$ ，传给 dsu 前减 1；查询输出 Y 或 N。

所需模板：dsu。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P3367>

```
1 int main()
2 {
3     int n, m;
4     scanf("%d%d", &n, &m);
5     dsu d(n);
6     while (m--)
7     {
8         int op, x, y;
9         scanf("%d%d%d", &op, &x, &y);
10        --x, --y;
11        if (op == 1)
12            d.merge(x, y);
13        else
14            puts(d.same(x, y) ? "Y" : "N");
15    }
16 }
```

2.1.2 可撤销并查集

RollbackDSU(n) 可使用 $0..n$ (若题目 0-based 则不用 n)。find 不压缩路径, merge 按集合大小合并, 已同集合返回 false 且不记历史; siz 仅根处有效。snapshot() 返回历史长度, 退出递归分支时 rollback(t) 撤销此后成功合并。t 必须是当前历史的祖先快照, 不能用已回滚分支的旧快照恢复版本。find/merge 为 $O(\log(n+1))$, 每次撤销成功合并 $O(1)$, 空间 $O(n)$; 适合版本树或时间分治。它按大小合并以保证回滚复杂度, 接口与普通 dsu 的根连接方向不同。

```

1 struct RollbackDSU
2 {
3     vector<int> fa, siz;
4     vector<pair<int, int>> his;
5
6     RollbackDSU(int n) : fa(n + 1), siz(n + 1, 1) { iota(fa.begin(), fa.end(), 0); }
7
8     int find(int x) const
9     {
10         while (x != fa[x]) x = fa[x];
11         return x;
12     }
13
14     int snapshot() const { return (int)his.size(); }
15
16     bool merge(int x, int y)
17     {
18         x = find(x);
19         y = find(y);
20         if (x == y) return false;
21         if (siz[x] < siz[y]) swap(x, y);
22         his.push_back({y, siz[x]});
23         fa[y] = x;
24         siz[x] += siz[y];
25         return true;
26     }
27
28     void rollback(int t)
29     {
30         assert(0 <= t && t <= snapshot());
31         while (snapshot() > t)
32         {
33             auto [y, s] = his.back();
34             his.pop_back();
35             siz[fa[y]] = s;
36             fa[y] = y;
37         }
38     }
39 };

```

模板题使用：Library Checker persistent_unionfind

题意：离线读入所有操作，按所依赖的版本建立树。版本 -1 映射到树根 0 ，操作 i 映射到 $i+1$ ；顶点仍是 0 -based。递归进入版本前保存 snapshot，退出时 rollback，答案按原查询顺序输出。只有合并产生可引用版本；本例是离线解法，不提供在线任意版本访问。总时间 $O(n+q \log(n+1))$ ，空间 $O(n+q)$ ，版本链递归深度可为 q 。

所需模板：RollbackDSU。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/persistent_unionfind

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, q;
6     cin >> n >> q;
7     vector<int> type(q + 1), u(q + 1), v(q + 1), ans(q + 1);
8     vector<vector<int>> g(q + 1);
9     for (int i = 1; i <= q; i++)
10    {
11        int k;
12        cin >> type[i] >> k >> u[i] >> v[i];
13        g[k + 1].push_back(i);
14    }
15    RollbackDSU d(n);
16    auto dfs = [&](auto &&self, int x) -> void
17    {
18        int t = d.snapshot();
19        if (x)
20        {
21            if (type[x] == 0)
22                d.merge(u[x], v[x]);
23            else
24                ans[x] = d.find(u[x]) == d.find(v[x]);
25        }
26        for (int y : g[x]) self(self, y);
27        d.rollback(t);
28    };
29    dfs(dfs, 0);
30    for (int i = 1; i <= q; i++)
31        if (type[i] == 1) cout << ans[i] << '\n';
32 }

```

第三章 堆

3.1 配对堆

3.1.1 PBDS 可修改、可合并优先队列

pheap<T,C> 默认比较器为 less<T>, 固定 pairing_heap_tag, 形成大根堆; 小根堆用 greater<T>。

包含 ext/pb_ds/priority_queue.hpp。push(x) 返回 point_iterator 句柄; modify(h,x) 改值, erase(h) 删除指定元素。top/pop 要求非空, size/empty/clear 沿用容器接口。a.join(b) 把 b 全部移入 a, b 变空; 原 b 元素句柄仍指向同一元素, 此后应交给 a。a.swap(b) 后句柄随元素交换归属。split(pred,b) 将满足谓词的元素移入 b, 本模板约定 b 原先为空。join/split 两个堆须不同, 比较规则一致。删除元素、pop 对应堆顶、clear 或销毁后, 相应句柄失效; 不要把原堆句柄用于深复制的副本。top、push、join 为 $O(1)$; pop 摊还 $O(\log n)$, modify/erase 的单次最坏可达 $O(n)$, 不当作最坏常数。split/复制/clear 可能扫描整个堆。适合需要修改已入堆元素或合并的任务; 只要普通入队出队时可用 std::priority_queue。复杂度按比较、复制标量或编号对为常数计, T 须满足容器的复制要求。

```
1 template <class T, class C = less<T>>
2 using pheap = __gnu_pbds::priority_queue<T, C, __gnu_pbds::pairing_heap_tag>;
```

```
1 pheap<int> a, b;
2 auto h = a.push(7);
3 a.modify(h, 2);
4 b.join(a);
5 b.erase(h);
```

3.2 左偏树

3.2.1 左偏树可并小根堆

默认构造为空节点池；`singleton(key)` 创建单点堆并返回节点编号，编号从 1 开始，0 表示空堆。`meld(x,y)` 原地合并两个不相交堆并返回新根，调用后旧根不能再当独立堆使用；`pop(root)` 删除非空堆的最小节点并返回新根，必须保存返回值。相同 `key` 按节点编号小者优先，`t[root].key` 为最小值，`root` 本身就是最小节点编号。`key` 支持完整 `long long`；仅比较而不做减法。`dist[空]=0`，叶子 `dist=1`，每点左子树 `dist` 不小于右子树；递归沿右链合并，`meld/pop` 为 $O(\log(n+1))$ ，其中 n 为参与堆节点数。`singleton` 均摊 $O(1)$ ，节点池空间 $O(\text{累计创建数})$ ，节点不回收，不保留版本；`pop` 后删除节点已脱离堆，不应继续作为根操作。只保证根操作，不提供任意点所属堆查询，P3377 驱动另用并查集和删除标记维护。不要改写内部 `l/r/dist`，也不要合并有交集的堆；`assert(x!=y)` 不能检测所有非法重叠。

```

1 struct LeftistHeap
2 {
3     struct Node
4     {
5         long long key;
6         int l, r, dist;
7     };
8
9     vector<Node> t{{0, 0, 0, 0}};
10
11     int singleton(long long key)
12     {
13         t.push_back({key, 0, 0, 1});
14         return int(t.size()) - 1;
15     }
16
17     int meld(int x, int y)
18     {
19         if (!x || !y) return x ? x : y;
20         assert(x != y);
21         if (pair(t[x].key, x) > pair(t[y].key, y)) swap(x, y);
22         t[x].r = meld(t[x].r, y);
23         if (t[t[x].l].dist < t[t[x].r].dist) swap(t[x].l, t[x].r);
24         t[x].dist = t[t[x].r].dist + 1;
25         return x;
26     }
27
28     int pop(int x)
29     {
30         assert(x > 0);
31         int root = meld(t[x].l, t[x].r);
32         t[x].l = t[x].r = 0;
33         t[x].dist = 1;
34         return root;
35     }
36 };

```

模板题使用：Luogu P3377

题意：合并指定元素所在的堆，或弹出其堆最小值；已删除元素的操作无效。同值先删先输入者。并查集代表不一定是堆根，root 单独保存；节点编号比输入数组下标大 1。

所需模板：LeftistHeap、dsu。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P3377>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m;
6     cin >> n >> m;
7     LeftistHeap heap;
8     dsu sets(n);
9     vector<int> root(n);
10    vector<bool> removed(n);
11    for (int i = 0; i < n; i++)
12    {
13        long long key;
14        cin >> key;
15        root[i] = heap.singleton(key);
16    }
17    while (m--)
18    {
19        int op, x;
20        cin >> op >> x;
21        x--;
22        if (op == 1)
23        {
24            int y;
25            cin >> y;
26            y--;
27            if (removed[x] || removed[y]) continue;
28            int a = sets.find(x), b = sets.find(y);
29            if (a == b) continue;
30            root[a] = heap.meld(root[a], root[b]);
31            sets.merge(a, b);
32        }
33        else if (removed[x])
34            cout << -1 << '\n';
35        else
36        {
37            int a = sets.find(x), id = root[a];
38            cout << heap.t[id].key << '\n';
39            removed[id - 1] = true;
40            root[a] = heap.pop(id);
41        }
42    }
43 }
```

第四章 树状数组

4.1 树状数组与第 k 小

$\text{add}(x,v)$ 、 $\text{sum}(x)$ 、 $\text{query}(l,r)$ ，下标 $1..n$ ，闭区间； $\text{kth}(k)$ 要求所有频率非负且 $k>0$ ，超过总和返回 $n+1$ 。单次 $O(\log n)$ 。T 须能从 0 构造并支持 $+=$ 与减法；可直接使用 `ModInt` 维护模数下的区间和。 kth 只适用于有序累计频数，不适用于取模后的区间和。

```
1 template <class T = long long> struct Fenwick
2 {
3     int n;
4     vector<T> a;
5
6     Fenwick(int n = 0) : n(n), a(n + 1) {}
7
8     void add(int x, T v)
9     {
10         assert(x >= 1);
11         for (; x <= n; x += x & -x) a[x] += v;
12     }
13
14     T sum(int x) const
15     {
16         T ans = 0;
17         for (; x; x -= x & -x) ans += a[x];
18         return ans;
19     }
20
21     T query(int l, int r) const { return sum(r) - sum(l - 1); }
22
23     // Nonnegative frequencies; returns n+1 if k exceeds total.
24     int kth(T k) const
25     {
26         assert(k > 0);
27         int x = 0, b = 1;
28         while (b <= n / 2) b *= 2;
29         for (; b; b >>= 1)
30             if (x + b <= n && a[x + b] < k)
31             {
32                 k -= a[x + b];
33                 x += b;
34             }
35         return x + 1;
36     }
37 };
```

第五章 线段树

5.1 线段树基础

5.1.1 通用懒标记线段树与边界搜索

模板参数依次为 S、F、op、e、mapping、composition、id；以数组 a 构造，0-based 半开区间；n 构造将 n 个元素置为 e。 $0 \leq n \leq 2^{28}-1$ ，受内存限制。op 满足结合律、e 为双侧单位元；单位标记不改变节点，mapping 必须保持 op 的合并结果；composition(f,g) 表示先 g 后 f，须满足作用复合一致性，id 为标记单位元。无需比较标记是否等于 id，每次下传都应用标记。set/get 单点，apply(l,r,f) 区间作用，prod(l,r) 保序合并，all() 读取根；空区间不更新且查询返回 e。区间长度若参与修改，必须存入 S，不能从 e 推断长度：区间和节点为 {sum,len}，真实叶 len=1，e={0,0}。max_right/min_left 的谓词须满足 ok(e())、无副作用、扩展时真变假后不再变真；两者查询会下传标记，非 const。递归深度 $O(\log n)$ ，构造 $O(n)$ 、操作/搜索 $O(\log n)$ 、all $O(1)$ ，空间 $O(n)$ ，假设运算和类型复制 $O(1)$ 。d/tag 及其他递归辅助方法仅供内部使用。

```
1 template <class S, class F, auto op, auto e, auto mapping, auto composition, auto id>
2 struct lazy_segtree
3 {
4     int n;
5     vector<S> d;
6     vector<F> tag;
7
8     lazy_segtree(int n = 0) : lazy_segtree(vector<S>(n, e())) {}
9
10    lazy_segtree(const vector<S> &a)
11        : n(a.size()), d(4 * n + 4, e()), tag(4 * n + 4, id())
12    {
13        if (n) build(1, 0, n, a);
14    }
15
16    void pull(int p) { d[p] = op(d[p * 2], d[p * 2 + 1]); }
17
18    void build(int p, int l, int r, const vector<S> &a)
19    {
20        if (r - l == 1)
21        {
22            d[p] = a[l];
23            return;
24        }
25        int m = (l + r) / 2;
26        build(p * 2, l, m, a);
27        build(p * 2 + 1, m, r, a);
28        pull(p);
29    }
```

```

31 void act(int p, F f)
32 {
33     d[p] = mapping(f, d[p]);
34     tag[p] = composition(f, tag[p]);
35 }
36
37 void push(int p)
38 {
39     act(p * 2, tag[p]);
40     act(p * 2 + 1, tag[p]);
41     tag[p] = id();
42 }
43
44 void set(int x, S v)
45 {
46     assert(0 <= x && x < n);
47     put(1, 0, n, x, v);
48 }
49
50 void put(int p, int l, int r, int x, S v)
51 {
52     if (r - l == 1)
53     {
54         d[p] = v;
55         tag[p] = id();
56         return;
57     }
58     push(p);
59     int m = (l + r) / 2;
60     if (x < m)
61         put(p * 2, l, m, x, v);
62     else
63         put(p * 2 + 1, m, r, x, v);
64     pull(p);
65 }
66
67 void apply(int l, int r, F f)
68 {
69     assert(0 <= l && l <= r && r <= n);
70     if (l < r) update(1, 0, n, l, r, f);
71 }
72
73 void update(int p, int l, int r, int a, int b, F f)
74 {
75     if (a <= l && r <= b)
76     {
77         act(p, f);
78         return;
79     }
80     push(p);
81     int m = (l + r) / 2;
82     if (a < m) update(p * 2, l, m, a, b, f);
83     if (m < b) update(p * 2 + 1, m, r, a, b, f);
84     pull(p);
85 }

```

```

87     S all() const { return d[1]; }
88
89     S get(int x)
90     {
91         assert(0 <= x && x < n);
92         return prod(x, x + 1);
93     }
94
95     S prod(int l, int r)
96     {
97         assert(0 <= l && l <= r && r <= n);
98         return l == r ? e() : query(1, 0, n, l, r);
99     }
100
101     S query(int p, int l, int r, int a, int b)
102     {
103         if (a <= l && r <= b) return d[p];
104         push(p);
105         int m = (l + r) / 2;
106         if (b <= m) return query(p * 2, l, m, a, b);
107         if (m <= a) return query(p * 2 + 1, m, r, a, b);
108         return op(query(p * 2, l, m, a, b), query(p * 2 + 1, m, r, a, b));
109     }
110
111     template <class G> int max_right(int l, G ok)
112     {
113         assert(0 <= l && l <= n && ok(e()));
114         S s = e();
115         return l == n ? n : right(1, 0, n, l, s, ok);
116     }
117
118     template <class G> int right(int p, int l, int r, int a, S &s, G &ok)
119     {
120         if (r <= a) return n;
121         if (a <= l && ok(op(s, d[p])))
122         {
123             s = op(s, d[p]);
124             return n;
125         }
126         if (r - l == 1) return l;
127         push(p);
128         int m = (l + r) / 2;
129         int x = right(p * 2, l, m, a, s, ok);
130         return x != n ? x : right(p * 2 + 1, m, r, a, s, ok);
131     }

```

```
133     template <class G> int min_left(int r, G ok)
134     {
135         assert(0 <= r && r <= n && ok(e()));
136         S s = e();
137         return r == 0 ? 0 : left(1, 0, n, r, s, ok);
138     }
139
140     template <class G> int left(int p, int l, int r, int b, S &s, G &ok)
141     {
142         if (b <= 1) return 0;
143         if (r <= b && ok(op(d[p], s)))
144         {
145             s = op(d[p], s);
146             return 0;
147         }
148         if (r - l == 1) return r;
149         push(p);
150         int m = (l + r) / 2;
151         int x = left(p * 2 + 1, m, r, b, s, ok);
152         return x != 0 ? x : left(p * 2, l, m, b, s, ok);
153     }
154 };
```

模板题使用：Library Checker range_affine_range_sum

题意：对数组区间施加 $x \rightarrow b * x + c$ ，并查询区间和。S 保存和与长度；空单位元长度 0，单个元素长度 1。composition(f,g) 表示先 g 后 f，mapping 按长度累加常数项。区间为 0-based 半开，模 998244353。

所需模板：lazy_segtree。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/range_affine_range_sum

```
1 using S = pair<long long, int>;
2 using F = pair<long long, long long>;
3 const int mod = 998244353;
4
5 S op(S a, S b)
6 {
7     return {(a.first + b.first) % mod, a.second + b.second};
8 }
9
10 S e()
11 {
12     return {0, 0};
13 }
14
15 S mapping(F f, S a)
16 {
17     return {(f.first * a.first + f.second * a.second) % mod, a.second};
18 }
19
20 F composition(F f, F g)
21 {
22     return {f.first * g.first % mod, (f.first * g.second + f.second) % mod};
23 }
24
25 F id()
26 {
27     return {1, 0};
28 }
```

使用入口（接上页配置函数）：

```

30 int main()
31 {
32     ios::sync_with_stdio(false);
33     cin.tie(nullptr);
34     int n, q;
35     cin >> n >> q;
36     vector<S> a(n);
37     for (auto &[x, len] : a)
38     {
39         cin >> x;
40         len = 1;
41     }
42     lazy_segtree<S, F, op, e, mapping, composition, id> s(a);
43     while (q--)
44     {
45         int t, l, r;
46         cin >> t >> l >> r;
47         if (t == 0)
48         {
49             long long b, c;
50             cin >> b >> c;
51             s.apply(l, r, {b, c});
52         }
53         else
54             cout << s.prod(l, r).first << '\n';
55     }
56 }

```

5.1.2 通用线段树与左右边界搜索

`segtree<S,op,e>(a)` 从 0-based vector 构建，或 `segtree(n)` 初始化为单位元。 $0 \leq n \leq 2^{29}$ ，实际受内存限制。`op(S,S)` 须满足结合律，`e()` 是双侧单位元；无需交换律。`set(p,x)`、`get(p)` 操作单点，`prod(l,r)` 返回半开区间 $[l,r)$ 从左到右的合并，空区间返回 `e`；`all()` 返回全区间结果。`max_right(l,ok)` 返回最大的 r 使 `ok(prod(l,r))` 为真；`min_left(r,ok)` 返回最小的 l 。要求 `ok(e())` 为真，`ok` 无副作用，并随对应方向扩展从真变假后不再变真；例如非负和不超过阈值，不能对任意负数和这样搜索。 $l=n$ 、 $r=0$ 支持空端点。构建 $O(n)$ ，单点/区间/边界搜索 $O(\log n)$ ，`all` $O(1)$ ，空间 $O(n)$ ，假设 `op`、`e`、`ok` 及 `S` 复制为 $O(1)$ 。`d`、`sz`、`pull` 为内部实现，不直接修改。函数复合时 `op(f,g)` 表示 `g(f(x))`，对应先执行左区间再执行右区间；`min_left` 不能照抄 `max_right` 的合并顺序。

```

1 template <class S, auto op, auto e> struct segtree
2 {
3     int n, sz = 1;
4     vector<S> d;
5
6     segtree(int n = 0) : segtree(vector<S>(n, e())) {}
7
8     segtree(const vector<S> &a) : n(a.size())
9     {
10         while (sz < n) sz <= 1;
11         d.assign(2 * sz, e());
12         copy(a.begin(), a.end(), d.begin() + sz);
13         for (int i = sz - 1; i; i--) pull(i);
14     }
15 }

```

```
16 void pull(int p) { d[p] = op(d[p * 2], d[p * 2 + 1]); }
17
18 void set(int p, S x)
19 {
20     assert(0 <= p && p < n);
21     d[p += sz] = x;
22     while (p >>= 1) pull(p);
23 }
24
25 S get(int p) const
26 {
27     assert(0 <= p && p < n);
28     return d[p + sz];
29 }
30
31 S all() const { return d[1]; }
```

```

33  S prod(int l, int r) const
34  {
35      assert(0 <= l && l <= r && r <= n);
36      S a = e(), b = e();
37      for (l += sz, r += sz; l < r; l >>= 1, r >>= 1)
38      {
39          if (l & 1) a = op(a, d[l++]);
40          if (r & 1) b = op(d[--r], b);
41      }
42      return op(a, b);
43  }
44
45  template <class F> int max_right(int l, F ok) const
46  {
47      assert(0 <= l && l <= n && ok(e()));
48      if (l == n) return n;
49      S a = e();
50      l += sz;
51      do
52      {
53          while (!(l & 1)) l >>= 1;
54          if (!ok(op(a, d[l])))
55          {
56              while (l < sz)
57              {
58                  l *= 2;
59                  if (ok(op(a, d[l]))) a = op(a, d[l++]);
60              }
61              return l - sz;
62          }
63          a = op(a, d[l++]);
64      } while ((l & -1) != 1);
65      return n;
66  }

```

```
68     template <class F> int min_left(int r, F ok) const
69     {
70         assert(0 <= r && r <= n && ok(e()));
71         if (r == 0) return 0;
72         S a = e();
73         r += sz;
74         do
75         {
76             --r;
77             while (r > 1 && (r & 1)) r >>= 1;
78             if (!ok(op(d[r], a)))
79             {
80                 while (r < sz)
81                 {
82                     r = r * 2 + 1;
83                     if (ok(op(d[r], a))) a = op(d[r--], a);
84                 }
85                 return r + 1 - sz;
86             }
87             a = op(d[r], a);
88         } while ((r & -r) != r);
89         return 0;
90     }
91 };
```

模板题使用：Library Checker point_set_range_composite

题意：单点修改一次函数，查询区间内从左到右依次作用于 x 的结果。 $op(f,g)$ 表示先 f 后 g ，单位元为 x ； $prod(l,r)$ 使用 0-based 半开区间，空区间返回单位元。配置函数须和 `main` 一起抄写。

所需模板：segtree。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：https://judge.yosupo.jp/problem/point_set_range_composite

```
1 using S = pair<long long, long long>;
2 const int mod = 998244353;
3
4 S op(S f, S g)
5 {
6     return {f.first * g.first % mod, (f.second * g.first + g.second) % mod};
7 }
8
9 S e()
10 {
11     return {1, 0};
12 }
13
14 int main()
15 {
16     ios::sync_with_stdio(false);
17     cin.tie(nullptr);
18     int n, q;
19     cin >> n >> q;
20     vector<S> a(n);
21     for (auto &[x, y] : a) cin >> x >> y;
22     segtree<S, op, e> s(a);
23     while (q--)
24     {
25         int t;
26         cin >> t;
27         if (t == 0)
28         {
29             int p;
30             long long c, d;
31             cin >> p >> c >> d;
32             s.set(p, {c, d});
33         }
34         else
35         {
36             int l, r;
37             long long x;
38             cin >> l >> r >> x;
39             auto [c, d] = s.prod(l, r);
40             cout << (c * x + d) % mod << '\n';
41         }
42     }
43 }
```

5.1.3 区间加与区间和

初值全零； $\text{add}(l,r,v)$ 、 $\text{query}(l,r)$ ， $1 \leq l \leq r \leq n$ 。维护 sum 与加法 tag ；所有中间和与长度乘积须在 long long 范围内。单次 $O(\log n)$ 。

```

1 struct LazySeg
2 {
3     int n;
4     vector<long long> sum, tag;
5
6     LazySeg(int n) : n(n), sum(4 * n + 4), tag(4 * n + 4) { assert(n > 0); }
7
8     void apply(int p, int l, int r, long long v)
9     {
10         sum[p] += (r - l + 1) * v;
11         tag[p] += v;
12     }
13
14     void push(int p, int l, int r)
15     {
16         int m = (l + r) / 2;
17         apply(p * 2, l, m, tag[p]);
18         apply(p * 2 + 1, m + 1, r, tag[p]);
19         tag[p] = 0;
20     }
21
22     void add(int l, int r, long long v) { add(1, 1, n, l, r, v); }
23
24     void add(int p, int l, int r, int ql, int qr, long long v)
25     {
26         if (ql <= l && r <= qr)
27         {
28             apply(p, l, r, v);
29             return;
30         }
31         push(p, l, r);
32         int m = (l + r) / 2;
33         if (ql <= m) add(p * 2, l, m, ql, qr, v);
34         if (qr > m) add(p * 2 + 1, m + 1, r, ql, qr, v);
35         sum[p] = sum[p * 2] + sum[p * 2 + 1];
36     }
37
38     long long query(int l, int r) { return query(1, 1, n, l, r); }
39
40     long long query(int p, int l, int r, int ql, int qr)
41     {
42         if (ql <= l && r <= qr) return sum[p];
43         push(p, l, r);
44         int m = (l + r) / 2;
45         long long ans = 0;
46         if (ql <= m) ans += query(p * 2, l, m, ql, qr);
47         if (qr > m) ans += query(p * 2 + 1, m + 1, r, ql, qr);
48         return ans;
49     }
50 };

```

5.1.4 线段树：区间乘、加、赋值与模意义求和

`AffineSegTree(a,mod)` 传入非空 0-based 初值数组；查询与修改采用 1-based 非空闭区间。`update(l,r,mul,add)` 将区间元素 x 改为 $mul \cdot x + add$ ；乘法用 $(mul, 0)$ ，加法用 $(1, add)$ ，赋值用 $(0, value)$ 。`query(l,r)` 返回区间和。 $1 \leq mod \leq INT_MAX$ ，模数无需为素数，初值和修改参数允许 long long 全范围，入口统一归一化。旧标记 (m, b) 后应用新标记 (a, c) ，复合为 $(m \cdot a, b \cdot a + c)$ ，乘法取模后再相加。建树 $O(n)$ ，单次 $O(\log n)$ ，空间 $O(n)$ 。内部 `build`、`apply` 等函数不作为公共接口；动态版重建新对象。

```

1 struct AffineSegTree
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         ll sum = 0, mul = 1, add = 0;
8     };
9
10    int n, mod;
11    vector<Node> t;
12
13    AffineSegTree(const vector<ll> &a, int mod) : n(a.size()), mod(mod), t(4 * n + 4)
14    {
15        assert(n > 0 && mod > 0);
16        build(1, 1, n, a);
17    }
18
19    ll norm(ll x) const
20    {
21        x %= mod;
22        if (x < 0) x += mod;
23        return x;
24    }
25
26    void pull(int p) { t[p].sum = (t[p * 2].sum + t[p * 2 + 1].sum) % mod; }
27
28    void build(int p, int l, int r, const vector<ll> &a)
29    {
30        t[p] = Node{};
31        if (l == r)
32        {
33            t[p].sum = norm(a[l - 1]);
34            return;
35        }
36        int m = (l + r) / 2;
37        build(p * 2, l, m, a);
38        build(p * 2 + 1, m + 1, r, a);
39        pull(p);
40    }
41
42    // New transform follows the old pending transform: x -> x*a+b.
43    void apply(int p, int len, ll a, ll b)
44    {
45        t[p].sum = (t[p].sum * a % mod + len * b % mod) % mod;
46        t[p].mul = t[p].mul * a % mod;
47        t[p].add = (t[p].add * a % mod + b) % mod;
48    }

```

```

49
50 void push(int p, int l, int r)
51 {
52     if (t[p].mul == 1 && t[p].add == 0) return;
53     int m = (l + r) / 2;
54     apply(p * 2, m - l + 1, t[p].mul, t[p].add);
55     apply(p * 2 + 1, r - m, t[p].mul, t[p].add);
56     t[p].mul = 1;
57     t[p].add = 0;
58 }
59
60 void update(int l, int r, ll a, ll b)
61 {
62     assert(1 <= l && l <= r && r <= n);
63     update(1, 1, n, l, r, norm(a), norm(b));
64 }
65
66 void update(int p, int l, int r, int ql, int qr, ll a, ll b)
67 {
68     if (ql <= l && r <= qr)
69     {
70         apply(p, r - l + 1, a, b);
71         return;
72     }
73     push(p, l, r);
74     int m = (l + r) / 2;
75     if (ql <= m) update(p * 2, l, m, ql, qr, a, b);
76     if (qr > m) update(p * 2 + 1, m + 1, r, ql, qr, a, b);
77     pull(p);
78 }
79
80 ll query(int l, int r)
81 {
82     assert(1 <= l && l <= r && r <= n);
83     return query(1, 1, n, l, r);
84 }
85
86 ll query(int p, int l, int r, int ql, int qr)
87 {
88     if (ql <= l && r <= qr) return t[p].sum;
89     push(p, l, r);
90     int m = (l + r) / 2;
91     ll answer = 0;
92     if (ql <= m) answer += query(p * 2, l, m, ql, qr);
93     if (qr > m) answer += query(p * 2 + 1, m + 1, r, ql, qr);
94     return answer % mod;
95 }
96 };

```

模板题使用：Luogu P3373

题意：对数组执行区间乘、区间加和区间求和，结果对给定模数取模。区间为 1-based 闭区间；update 的两个参数表示先乘再加。

所需模板：AffineSegTree。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P3373>

```
1 int main()
2 {
3     std::ios::sync_with_stdio(false);
4     std::cin.tie(nullptr);
5     int n, q, mod;
6     std::cin >> n >> q >> mod;
7     std::vector<long long> a(n);
8     for (auto &x : a) std::cin >> x;
9     AffineSegTree tree(a, mod);
10    while (q--)
11    {
12        int op, l, r;
13        std::cin >> op >> l >> r;
14        if (op == 3)
15            std::cout << tree.query(l, r) << '\n';
16        else
17        {
18            long long k;
19            std::cin >> k;
20            if (op == 1)
21                tree.update(l, r, k, 0);
22            else
23                tree.update(l, r, 1, k);
24        }
25    }
26 }
```

相关应用：线段树维护单调栈的双向极大值贡献；区间加接雨水应用，不将普通单调栈直接视为覆盖。

5.1.5 线段树维护单调栈：区间加与接雨水

`MonotoneStackSeg(a)` 要求非空 `long long` 数组；`add(l,r,delta)` 对 0-based 半开区间加值，空区间不变。`water()` 返回整个数组的接雨水体积。`scan(l,r,height,reverse=false)` 按左到右 (`reverse=true` 则右到左) 扫描，将 `height` 与当前元素取 `max` 并累加 `height-a[i]`，返回 (累加值, 最后的 `height`)；空区间返回 (0,height)。支持负高度与负增量，无固定零边界。节点缓存双向前缀极大值减原值的和；区间平移不改变两者差。建树 $O(n)$ ，`add/scan` 最坏 $O(\log^2(n+1))$ ，`water` 为 $O(1)$ ，空间 $O(n)$ ；递归深度 $O(\log n)$ 。 $4n$ 须可由 `int` 表示；高度、和、懒标记、外部 `height` 与长度的乘积以及所有中间运算均须在 `long long` 范围内。`t` 与 `extra/pull/push` 及带节点编号的函数属内部实现。

```

1 struct MonotoneStackSeg
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int len = 0;
8         ll mx = 0, sum = 0, lazy = 0;
9         array<ll, 2> fill{};
10    };
11
12    int n;
13    vector<Node> t;
14
15    MonotoneStackSeg(const vector<ll> &a) : n(a.size()), t(4 * n)
16    {
17        assert(n > 0);
18        build(1, 0, n, a);
19    }
20
21    void apply(int p, ll delta)
22    {
23        t[p].mx += delta;
24        t[p].sum += delta * t[p].len;
25        t[p].lazy += delta;
26    }
27
28    void push(int p)
29    {
30        if (!t[p].lazy) return;
31        apply(p * 2, t[p].lazy);
32        apply(p * 2 + 1, t[p].lazy);
33        t[p].lazy = 0;
34    }

```

单路扫描、合并与建树（接上页同一结构体）：

```

36  ll extra(int p, ll height, int dir)
37  {
38      if (height >= t[p].mx) return height * t[p].len - t[p].sum;
39      if (t[p].len == 1) return 0;
40      push(p);
41      int first = p * 2 + dir, second = p * 2 + 1 - dir;
42      if (height >= t[first].mx)
43          return height * t[first].len - t[first].sum + extra(second, height, dir);
44      return extra(first, height, dir) + (t[p].fill[dir] - t[first].fill[dir]);
45  }
46
47  void pull(int p)
48  {
49      int l = p * 2, r = l + 1;
50      t[p].len = t[l].len + t[r].len;
51      t[p].mx = max(t[l].mx, t[r].mx);
52      t[p].sum = t[l].sum + t[r].sum;
53      t[p].fill[0] = t[l].fill[0] + extra(r, t[l].mx, 0);
54      t[p].fill[1] = t[r].fill[1] + extra(l, t[r].mx, 1);
55  }
56
57  void build(int p, int l, int r, const vector<ll> &a)
58  {
59      if (r - l == 1)
60      {
61          t[p].len = 1;
62          t[p].mx = a[l];
63          t[p].sum = a[l];
64          return;
65      }
66      int m = (l + r) / 2;
67      build(p * 2, l, m, a);
68      build(p * 2 + 1, m, r, a);
69      pull(p);
70  }

```

区间加、双向查询与雨水体积（接上页同一结构体）：

```

72 void add(int p, int l, int r, int ql, int qr, ll delta)
73 {
74     if (ql <= l && r <= qr) return apply(p, delta);
75     push(p);
76     int m = (l + r) / 2;
77     if (ql < m) add(p * 2, l, m, ql, qr, delta);
78     if (m < qr) add(p * 2 + 1, m, r, ql, qr, delta);
79     pull(p);
80 }
81
82 void add(int l, int r, ll delta)
83 {
84     assert(0 <= l && l <= r && r <= n);
85     if (l < r) add(1, 0, n, l, r, delta);
86 }
87
88 ll scan(int p, int l, int r, int ql, int qr, ll &height, int dir)
89 {
90     if (ql <= l && r <= qr)
91     {
92         ll answer = extra(p, height, dir);
93         height = max(height, t[p].mx);
94         return answer;
95     }
96     push(p);
97     int m = (l + r) / 2;
98     ll answer = 0;
99     if (dir && m < qr) answer += scan(p * 2 + 1, m, r, ql, qr, height, dir);
100    if (ql < m) answer += scan(p * 2, l, m, ql, qr, height, dir);
101    if (!dir && m < qr) answer += scan(p * 2 + 1, m, r, ql, qr, height, dir);
102    return answer;
103 }
104
105 // Return (sum of running maximum - a[i], final running maximum).
106 pair<ll, ll> scan(int l, int r, ll height, bool reverse = false)
107 {
108     assert(0 <= l && l <= r && r <= n);
109     ll answer = 0;
110     if (l < r) answer = scan(1, 0, n, l, r, height, reverse);
111     return {answer, height};
112 }
113
114 ll water() const
115 {
116     ll gap = t[1].mx * n - t[1].sum;
117     return t[1].fill[0] - (gap - t[1].fill[1]);
118 }
119 };

```

应用题补充：NERC 2023 G / Luogu P12438

题意：Great City Saint Petersburg：先输出初始雨水量，每次输入 1-based 闭区间使高度加一，再输出全局雨水量。调用时只减 1。该题为比赛应用，不计正式模板题覆盖；一般负增量和 scan 接口另有本地验证。

所需模板：MonotoneStackSeg。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P12438>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, q;
6     cin >> n >> q;
7     vector<long long> a(n);
8     for (auto &x : a) cin >> x;
9     MonotoneStackSeg s(a);
10    cout << s.water() << '\n';
11    while (q--)
12    {
13        int l, r;
14        cin >> l >> r;
15        s.add(l - 1, r, 1);
16        cout << s.water() << '\n';
17    }
18 }
```

应用题补充：带外部高度的双向扫描（本地接口示例）

题意：输入 n, q 和高度数组，再输入 op, l, r, x ；区间均为 0-based 半开。 $op=0$ 对区间加 x ； $op=1$ 从左向右、 $op=2$ 从右向左，初始最高值为 x ，输出逐步最高值与当前元素之差的和，以及最终最高值。空区间保持 x ，支持负数。

所需模板：MonotoneStackSeg。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://github.com/gsh20040816/CPCTemplate/issues/11>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, q;
6     cin >> n >> q;
7     vector<long long> a(n);
8     for (auto &x : a) cin >> x;
9     MonotoneStackSeg s(a);
10    while (q--)
11    {
12        int op, l, r;
13        long long x;
14        cin >> op >> l >> r >> x;
15        if (op == 0)
16            s.add(l, r, x);
17        else
18        {
19            auto [cost, height] = s.scan(l, r, x, op == 2);
20            cout << cost << ' ' << height << '\n';
21        }
22    }
23 }
```

5.2 李超线段树

5.2.1 精确线段李超树：最大值编号

构造传入非空的所有查询横坐标，排序去重后数量 M 。add(x_1, y_1, x_2, y_2) 插入闭线段并返回从 1 开始的编号；可交换端点，竖直线段在该横坐标取较高端点。query(x) 返回交点最高的线段编号，同值取较早插入者；无交点返回 0, x 必须在预登记坐标内。value(id, x) 返回未约分的 (分子, 正分母), 要求 $id > 0$ 且 x 在线段内；不直接相除即可保留精度。横坐标绝对值 $\leq 10^9$, 端点纵坐标绝对值 $\leq 2 \times 10^{18}$, int128 交叉乘积在这些界内安全；不使用浮点或 epsilon。最小值问题可先把纵坐标取负，得到编号后再还原。整条直线用原 LiChao 更短；这里插入仅作用于端点范围。构造 $O(M \log M)$, 插入 $O(\log^2(M+1))$, 查询 $O(\log(M+1))$, 空间 $O(M + \text{线段数})$; 要求 $M < \text{INT_MAX}/4$, 累计插入线段数 $< \text{INT_MAX}$ 。

```

1 struct SegmentLiChao
2 {
3     using ll = long long;
4     using I = __int128_t;
5
6     struct Line
7     {
8         ll x1, y1, x2, y2;
9     };
10
11     vector<ll> xs;
12     vector<Line> lines{{0, 0, 0, 0}};
13     vector<int> t;
14
15     SegmentLiChao(vector<ll> x) : xs(move(x))
16     {
17         sort(xs.begin(), xs.end());
18         xs.erase(unique(xs.begin(), xs.end()), xs.end());
19         assert(!xs.empty() && xs.front() >= -10000000000 && xs.back() <= 10000000000);
20         t.resize(4 * xs.size());
21     }
22
23     pair<I, I> value(int id, ll x) const
24     {
25         auto [a, b, c, d] = lines[id];
26         if (a == c) return {max(b, d), 1};
27         return {I(b) * (c - x) + I(d) * (x - a), c - a};
28     }

```

精确比较与线段插入（接上页同一结构体）：

```

30  bool better(int u, int v, ll x) const
31  {
32      if (!u || !v) return u != 0;
33      auto [a, b] = value(u, x);
34      auto [c, d] = value(v, x);
35      return a * d == c * b ? u < v : a * d > c * b;
36  }
37
38  void insert(int p, int l, int r, int ql, int qr, int id)
39  {
40      int m = (l + r) / 2;
41      if (ql <= l && r <= qr)
42      {
43          if (better(id, t[p], xs[m])) swap(id, t[p]);
44          if (!id || l == r) return;
45          if (better(id, t[p], xs[l]))
46              insert(p * 2, l, m, l, m, id);
47          else if (better(id, t[p], xs[r]))
48              insert(p * 2 + 1, m + 1, r, m + 1, r, id);
49          return;
50      }
51      if (ql <= m) insert(p * 2, l, m, ql, qr, id);
52      if (qr > m) insert(p * 2 + 1, m + 1, r, ql, qr, id);
53  }
54
55  int add(ll x1, ll y1, ll x2, ll y2)
56  {
57      assert(-10000000000 <= x1 && x1 <= 10000000000);
58      assert(-10000000000 <= x2 && x2 <= 10000000000);
59      assert(-2000000000000000000LL <= y1 && y1 <= 2000000000000000000LL);
60      assert(-2000000000000000000LL <= y2 && y2 <= 2000000000000000000LL);
61      if (x1 > x2)
62      {
63          swap(x1, x2);
64          swap(y1, y2);
65      }
66      int id = lines.size();
67      lines.push_back({x1, y1, x2, y2});
68      int l = lower_bound(xs.begin(), xs.end(), x1) - xs.begin();
69      int r = upper_bound(xs.begin(), xs.end(), x2) - xs.begin() - 1;
70      if (l <= r) insert(1, 0, (int)xs.size() - 1, l, r, id);
71      return id;
72  }

```

查询最优线段编号（接上页同一结构体）：

```
74     int query(ll x) const
75     {
76         int i = lower_bound(xs.begin(), xs.end(), x) - xs.begin();
77         assert(i < (int)xs.size() && xs[i] == x);
78         int p = 1, l = 0, r = (int)xs.size() - 1, ans = 0;
79         while (true)
80         {
81             if (better(t[p], ans, x)) ans = t[p];
82             if (l == r) return ans;
83             int m = (l + r) / 2;
84             if (i <= m)
85             {
86                 p *= 2;
87                 r = m;
88             }
89             else
90             {
91                 p = p * 2 + 1;
92                 l = m + 1;
93             }
94         }
95     }
96 };
```

模板题使用：Library Checker segment_add_get_min

题意：在线段有效区间内查询最小值。预读所有查询横坐标以离散化，再按原序处理；没有查询时直接退出。题目横坐标都是整数，因此半开区间 $[l, r)$ 换成闭区间 $[l, r-1]$ 。端点值取负，把最小值转为最大值；无结果输出 INFINITY，返回分数在本题必为整数。原线性式的端点计算在题目范围内可用 long long。

所需模板：SegmentLiChao。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/segment_add_get_min

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, q;
6     cin >> n >> q;
7
8     struct Query
9     {
10         int type;
11         long long l, r, a, b;
12     };
13
14     vector<Query> ops(n + q);
15     vector<long long> xs;
16     for (int i = 0; i < n + q; i++)
17     {
18         auto &o = ops[i];
19         o.type = 0;
20         if (i >= n) cin >> o.type;
21         if (o.type == 0)
22             cin >> o.l >> o.r >> o.a >> o.b;
23         else
24         {
25             cin >> o.l;
26             xs.push_back(o.l);
27         }
28     }
```

继续处理操作（接上页同一 main）：

```
29     if (xs.empty()) return 0;
30     SegmentLiChao t(xs);
31     for (auto o : ops)
32         if (o.type == 0)
33             t.add(o.l, -o.a * o.l - o.b, o.r - 1, -o.a * (o.r - 1) - o.b);
34     else
35     {
36         int id = t.query(o.l);
37         if (!id)
38             cout << "INFINITY\n";
39         else
40         {
41             auto [num, den] = t.value(id, o.l);
42             cout << (long long)(-num / den) << '\n';
43         }
44     }
45 }
```

应用题补充：Luogu P4097 / HEOI2013 Segment

题意：强制在线插入整数端点线段，查询最高交点的最早线段编号。预登记全部 1..39989 横坐标；每个操作用上次查询答案解码，插入不会修改 last。竖直线段取较高端点，无交点输出并保存 0。此题源自 HEOI，作为应用补充单列。

所需模板：SegmentLiChao。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P4097>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     vector<long long> xs(39989);
6     iota(xs.begin(), xs.end(), 1);
7     SegmentLiChao t(xs);
8     int q, last = 0;
9     cin >> q;
10    while (q--)
11    {
12        int op, x1, x2;
13        long long y1, y2;
14        cin >> op >> x1;
15        x1 = (x1 + last - 1) % 39989 + 1;
16        if (op == 0)
17        {
18            last = t.query(x1);
19            cout << last << '\n';
20        }
21        else
22        {
23            cin >> y1 >> x2 >> y2;
24            x2 = (x2 + last - 1) % 39989 + 1;
25            y1 = (y1 + last - 1) % 1000000000 + 1;
26            y2 = (y2 + last - 1) % 1000000000 + 1;
27            t.add(x1, y1, x2, y2);
28        }
29    }
30 }
```

5.2.2 离散李超树与最优直线编号

构造传入所有可能查询的横坐标，`add({k,b,id})` 插入直线，`query(x)` 返回最小值与最小编号，要求 `x` 在预登记坐标内且至少有一条线。整数求值用 `int128`，`k`、`b`、`x` 为有符号 64 位且中间值可表示。单次 $O(\log n)$ 。

```

1 struct LiChao
2 {
3     using ll = long long;
4     using I = __int128_t;
5
6     struct Line
7     {
8         ll k = 0, b = 0;
9         int id = -1;
10
11         I value(ll x) const { return I(k) * x + b; }
12     };
13
14     struct Node
15     {
16         Line line;
17         int left = -1, right = -1;
18     };
19
20     vector<ll> xs;
21     vector<Node> a{Node{}};
22
23     LiChao(vector<ll> x) : xs(x)
24     {
25         sort(xs.begin(), xs.end());
26         xs.erase(unique(xs.begin(), xs.end()), xs.end());
27         assert(!xs.empty());
28     }
29
30     bool better(Line u, Line v, ll x) const
31     {
32         return v.id == -1 ||
33             (u.id != -1 && pair{u.value(x), u.id} < pair{v.value(x), v.id});
34     }
35
36     void add(Line line)
37     {
38         assert(line.id >= 0);
39         add(0, 0, (int)xs.size() - 1, line);
40     }

```

插入与查询（接上页同一结构体）：

```

42 void add(int p, int l, int r, Line line)
43 {
44     int m = (l + r) / 2;
45     if (better(line, a[p].line, xs[m])) swap(line, a[p].line);
46     if (l == r || line.id == -1) return;
47     bool left = better(line, a[p].line, xs[l]);
48     bool right = better(line, a[p].line, xs[r]);
49     if (!left && !right) return;
50     int next = left ? a[p].left : a[p].right;
51     if (next == -1)
52     {
53         next = (int)a.size();
54         a.push_back(Node{});
55         if (left)
56             a[p].left = next;
57         else
58             a[p].right = next;
59     }
60     if (left)
61         add(next, l, m, line);
62     else
63         add(next, m + 1, r, line);
64 }
65
66 pair<I, int> query(ll x) const
67 {
68     int idx = lower_bound(xs.begin(), xs.end(), x) - xs.begin();
69     assert(idx < (int)xs.size() && xs[idx] == x);
70     int p = 0, l = 0, r = (int)xs.size() - 1;
71     Line best;
72     while (p != -1)
73     {
74         if (better(a[p].line, best, x)) best = a[p].line;
75         if (l == r) break;
76         int m = (l + r) / 2;
77         if (idx <= m)
78         {
79             p = a[p].left;
80             r = m;
81         }
82         else
83         {
84             p = a[p].right;
85             l = m + 1;
86         }
87     }
88     assert(best.id != -1);
89     return {best.value(x), best.id};
90 }
91 };

```

模板题使用：Library Checker line_add_get_min

题意：插入整条直线并查询最小值。先收集查询横坐标，再按输入顺序处理；初始至少一条线，查询必有结果。编号由调用者递增指定，本题只输出值。题目系数与横坐标范围保证最终值可转为 long long；模板内部用 int128 比较。

所需模板：LiChao。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/line_add_get_min

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, q;
6     cin >> n >> q;
7
8     struct Query
9     {
10         int type;
11         long long a, b;
12     };
13
14     vector<Query> ops(n + q);
15     vector<long long> xs;
16     for (int i = 0; i < n + q; i++)
17     {
18         auto &o = ops[i];
19         o.type = 0;
20         if (i >= n) cin >> o.type;
21         cin >> o.a;
22         if (o.type == 0)
23             cin >> o.b;
24         else
25             xs.push_back(o.a);
26     }
27     if (xs.empty()) return 0;
28     LiChao t(xs);
29     int id = 0;
30     for (auto o : ops)
31         if (o.type == 0)
32             t.add({o.a, o.b, id++});
33         else
34             cout << (long long)t.query(o.a).first << '\n';
35 }
```

第六章 二叉搜索树 & 平衡树

6.1 二叉搜索树 & 平衡树

6.1.1 GNU rope 与版本共享

实际类型为 `__gnu_cxx::rope<T>`, 不是 `std::rope`; 它是 GNU 扩展序列, 包含 `ext/rope`。本工具链随后重新包含 `cassert`, 保留断言接口。`rp<int>`、`rp<char>` 表示序列, 默认空。`size()`、`empty()`、`push_back(x)`、`insert(p,x)`、`erase(p,len)`、`replace(p,x)`、`substr(p,len)` 直接沿用 `rope`; 位置为 0-based, `len` 是长度而非右端点。约定 $0 \leq p \leq \text{size}$, 删除/截取要求 $\text{len} \leq \text{size} - p$, 读写单点要求 $p < \text{size}$ 。`r[i]` 读取第 i 项; `substr` 返回新序列。`rp<T> b=a` 共享底层树, 随后通过容器接口修改 `b` 不影响 `a`, 可用 `vector<rp<int>` 保存分支版本。复制构造共享根为 $O(1)$; 随机访问沿树路径, 正常平衡时 $O(\log n)$ 。插入/删除/截取涉及路径、叶块复制和可能的再平衡, 不能把每次操作都写成严格最坏 $O(\log n)$; 释放最后引用可能递归回收整棵子树。长期保留所有版本的内存不能按单个序列 $O(n)$ 估计。不可缓存修改前的迭代器或使用可变代理跨版本写入。仅使用默认内存分配器及普通整数/字符值; 函数生成型 `rope` 不在本模板范围。可持久化队列已通过定长构造加头尾下标版本: Q 个零单元, 复制根与 l/r , 入队 `replace(r++,x)`, 出队读取 `[l++]`; 直接 `push_back/erase` 版本在针对数据上耗尽内存。线上 942 ms、591.26 MiB, 具体失败与通过范围见 `docs/ROPE-NOTES.md`。适合需要序列中间插删、截取和少量分支快照的场景; 只按下标读写通常使用 `vector` 更直接。需要区间和、翻转等懒标记聚合时使用隐式 `Treap`, `rope` 本身不提供这些接口。下方示例通过修改 `b` 演示 `a` 保持原版本。

只需键值查改而不需要有序排名时, `gp_hash_table` / `cc_hash_table` 及选型建议见第 1 页。

```
1 template <class T> using rp = __gnu_cxx::rope<T>;
```

```
1 rp<int> a;  
2 a.push_back(3);  
3 a.insert(0, 7);  
4 rp<int> b = a;  
5 b.replace(1, 9);  
6 b.erase(0, 1);  
7 auto c = a.substr(0, 1);  
8 int x = a[1];
```

模板题使用：Library Checker persistent_queue

题意：维护可持久化队列，每次从指定版本入队或出队并建立新版本。题目的 -1 号空版本映射为下标 0；rope 副本和头尾指针一起复制，replace 写入不会改变历史版本，出队只推进该版本队头。

所需模板：rp。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/persistent_queue

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int q;
6     cin >> q;
7     vector<rp<int>> s(q + 1);
8     vector<int> l(q + 1), r(q + 1);
9     s[0] = rp<int>(q, 0);
10    for (int i = 1; i <= q; i++)
11    {
12        int op, t, x;
13        cin >> op >> t;
14        ++t;
15        s[i] = s[t];
16        l[i] = l[t];
17        r[i] = r[t];
18        if (op == 0)
19        {
20            cin >> x;
21            s[i].replace(r[i]++, x);
22        }
23        else
24        {
25            cout << s[i][l[i]++] << '\n';
26        }
27    }
28 }
```

6.1.2 PBDS 有序树与顺序统计

GNU libstdc++ 扩展, 包含 `ext/pb_ds/assoc_container.hpp` 和 `ext/pb_ds/tree_policy.hpp`。 `ost<T>` 为去重有序集合; `insert(x)`, `erase(x)`, `find(x)`, `lower_bound(x)`, `upper_bound(x)` 沿用集合接口。 `order_of_key(x)` 为严格小于 `x` 的元素数, `find_by_order(k)` 返回 0-based 第 `k` 个的迭代器, 越界返回 `end`。默认 `less<T>`; 改用其他严格弱序比较器时, 排名和边界均按该比较器解释。单次 $O(\log n)$, 空间 $O(n)$, 假设比较为 $O(1)$ 。重复值使用 `ost<pair<T,int>`, 插入 `{x,id}` 且 `id` 全局唯一、 $1 \leq id < INT_MAX$ 。 `order_of_key({x,0})` 统计小于 `x`, `order_of_key({x,INT_MAX})` 统计不大于 `x`; `find_by_order(k)->first` 取值, 须先检查 `end`。删除一个 `x`: `it=lower_bound({x,0})`, 检查 `it!=end` 且 `it->first==x` 后 `erase(it)`。不可用 `less_equal` 模拟多重集合。前驱先检查迭代器是否为 `begin`, 再递减; 空集合和无解不得解引用 `end`。

只需键值查改而不需要有序排名时, `gp_hash_table` / `cc_hash_table` 及选型建议见第 1 页。

序列中间插删、截取与版本共享的 GNU rope 见第 40 页。

```

1 template <class T, class C = less<T>>
2 using ost = __gnu_pbds::tree<T,
3
4         __gnu_pbds::null_type,
5         C,
6         __gnu_pbds::rb_tree_tag,
7         __gnu_pbds::tree_order_statistics_node_update>;

```

模板题使用：Library Checker `ordered_set`

题意：维护无重复整数集合，支持第 k 小、计数及含等号的前驱/后继。`find_by_order` 使用 0-based 排名；`order_of_key` 只计严格小于 x ，故不大于 x 的数量需补上相等元素。越界查询按题意输出 -1。

所需模板：`ost`。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：https://judge.yosupo.jp/problem/ordered_set

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, q, x;
6     cin >> n >> q;
7     ost<int> s;
8     while (n--)
9     {
10         cin >> x;
11         s.insert(x);
12     }
13     while (q--)
14     {
15         int op;
16         cin >> op >> x;
17         if (op == 0) s.insert(x);
18         if (op == 1) s.erase(x);
19         if (op == 2)
20         {
21             auto it = s.find_by_order(x - 1);
22             cout << (it == s.end() ? -1 : *it) << '\n';
23         }
24         if (op == 3) cout << s.order_of_key(x) + (s.find(x) != s.end()) << '\n';
25         if (op == 4)
26         {
27             auto it = s.upper_bound(x);
28             cout << (it == s.begin() ? -1 : *--it) << '\n';
29         }
30         if (op == 5)
31         {
32             auto it = s.lower_bound(x);
33             cout << (it == s.end() ? -1 : *it) << '\n';
34         }
35     }
36 }
```

6.2 Treap

6.2.1 FHQ Treap: 可重集合与顺序统计

insert(x) 插入一个副本, erase(x) 删除一个副本并返回是否存在; rank(x) 为严格小于 x 的元素数加一, x 不必存在; kth(k) 要求 $1 \leq k \leq \text{size}()$ 。prev/next 为严格前驱/后继, 不存在时返回 nullopt。支持重复值、LLONG_MIN 和 LLONG_MAX, 分裂不使用 x-1。随机优先级下单次期望 $O(\log n)$, 并非确定性最坏界。节点池不回收已删除节点, 空间按累计插入次数计算; 可在构造时指定随机种子。

```

1 struct OrderedTreap
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int l = 0, r = 0, siz = 0;
8         ll val = 0;
9         unsigned long long pri = 0;
10    };
11
12    vector<Node> a{Node{}};
13    mt19937_64 rng;
14    int root = 0;
15
16    OrderedTreap(unsigned long long seed = 712367821) : rng(seed) {}
17
18    void pull(int p) { a[p].siz = a[a[p].l].siz + a[a[p].r].siz + 1; }
19
20    int node(ll x)
21    {
22        int p = a.size();
23        a.push_back({0, 0, 1, x, rng()});
24        return p;
25    }
26
27    // left: val < x, or val <= x when equal is true. No x-1 overflow.
28    void split(int p, ll x, int &l, int &r, bool equal = false)
29    {
30        if (!p)
31        {
32            l = r = 0;
33            return;
34        }
35        if (a[p].val < x || (equal && a[p].val == x))
36        {
37            l = p;
38            split(a[p].r, x, a[p].r, r, equal);
39        }
40        else
41        {
42            r = p;
43            split(a[p].l, x, l, a[p].l, equal);
44        }
45        pull(p);
46    }
47

```

```

48 int merge(int l, int r)
49 {
50     if (!l || !r) return l ? l : r;
51     if (a[l].pri > a[r].pri)
52     {
53         a[l].r = merge(a[l].r, r);
54         pull(l);
55         return l;
56     }
57     a[r].l = merge(l, a[r].l);
58     pull(r);
59     return r;
60 }
61
62 void insert(ll x)
63 {
64     int p = node(x), l, r;
65     split(root, x, l, r);
66     root = merge(merge(l, p), r);
67 }
68
69 bool erase(ll x)
70 {
71     int l, mid, r;
72     split(root, x, l, mid);
73     split(mid, x, mid, r, true);
74     bool found = mid != 0;
75     if (mid) mid = merge(a[mid].l, a[mid].r);
76     root = merge(l, merge(mid, r));
77     return found;
78 }
79
80 int size() const { return a[root].siz; }
81
82 int less(ll x, bool equal = false) const
83 {
84     int p = root, ans = 0;
85     while (p)
86     {
87         if (a[p].val < x || (equal && a[p].val == x))
88         {
89             ans += a[a[p].l].siz + 1;
90             p = a[p].r;
91         }
92         else
93             p = a[p].l;
94     }
95     return ans;
96 }
97
98 int rank(ll x) const { return less(x) + 1; }
99
100 ll kth(int k) const
101 {
102     assert(1 <= k && k <= size());
103     int p = root;

```

```
104     while (true)
105     {
106         int left = a[a[p].l].siz;
107         if (k == left + 1) return a[p].val;
108         if (k <= left)
109             p = a[p].l;
110         else
111         {
112             k -= left + 1;
113             p = a[p].r;
114         }
115     }
116 }
117
118 optional<ll> prev(ll x) const
119 {
120     int k = less(x);
121     if (!k) return nullopt;
122     return kth(k);
123 }
124
125 optional<ll> next(ll x) const
126 {
127     int k = less(x, true);
128     if (k == size()) return nullopt;
129     return kth(k + 1);
130 }
131 };
```

模板题使用：Luogu P3369

题意：维护可重复整数集合，支持插入、删除一个值、排名、第 k 小、严格前驱和严格后继。题目保证前驱/后继存在，因此可解引用返回值。

所需模板：OrderedTreap。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P3369>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n;
6     cin >> n;
7     OrderedTreap g;
8     while (n--)
9     {
10         int op;
11         long long x;
12         cin >> op >> x;
13         if (op == 1)
14             g.insert(x);
15         else if (op == 2)
16             g.erase(x);
17         else if (op == 3)
18             cout << g.rank(x) << '\n';
19         else if (op == 4)
20             cout << g.kth(x) << '\n';
21         else if (op == 5)
22             cout << *g.prev(x) << '\n';
23         else
24             cout << *g.next(x) << '\n';
25     }
26 }
```

6.2.2 隐式 Treap：顺序反转、位翻转、加值与区间和

`insert(k,value)` 在前 k 个元素后插入， $0 \leq k \leq \text{size}()$ ；区间接口 `erase`、`reverse`、`flip_bits`、`add`、`query` 均使用 1-based 非空闭区间。`reverse` 反转元素顺序；`flip_bits` 要求区间内全为 0/1，逐位取反并保持顺序，不检查这一前提。`query` 返回和，二进制区间即 1 的个数。`values()` 递归中序返回完整序列， $O(n)$ 。其余操作期望 $O(\log n)$ ，不是最坏界。符号与加法标记表示 $s*x+b$ ，位翻转通过取负后加 1 实现，下传必须先符号再加法；顺序反转另行维护。所有值、和、标记、乘法及取负中间结果须在 `long long` 范围内。删除不回收节点，空间按累计插入次数计算；辅助函数和节点数组只供内部使用。

```

1 struct SequenceTreap
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int l = 0, r = 0, siz = 0;
8         ll val = 0, sum = 0, tag = 0;
9         unsigned long long pri = 0;
10        bool rev = false, neg = false;
11    };
12
13    vector<Node> a{Node{}};
14    mt19937_64 rng;
15    int root = 0;
16
17    SequenceTreap(unsigned long long seed = 712367821) : rng(seed) {}
18
19    int size() const { return a[root].siz; }
20
21    void pull(int p)
22    {
23        a[p].siz = a[a[p].l].siz + a[a[p].r].siz + 1;
24        a[p].sum = a[a[p].l].sum + a[p].val + a[a[p].r].sum;
25    }
26
27    void apply_add(int p, ll x)
28    {
29        if (!p) return;
30        a[p].val += x;
31        a[p].sum += x * a[p].siz;
32        a[p].tag += x;
33    }
34
35    void apply_negate(int p)
36    {
37        if (!p) return;
38        a[p].val = -a[p].val;
39        a[p].sum = -a[p].sum;
40        a[p].tag = -a[p].tag;
41        a[p].neg = !a[p].neg;
42    }

```

顺序反转、懒标记下传与分裂（接上页同一结构体）：

```

44 void apply_reverse(int p)
45 {
46     if (!p) return;
47     swap(a[p].l, a[p].r);
48     a[p].rev = !a[p].rev;
49 }
50
51 void push(int p)
52 {
53     if (a[p].rev)
54     {
55         apply_reverse(a[p].l);
56         apply_reverse(a[p].r);
57         a[p].rev = false;
58     }
59     if (a[p].neg)
60     {
61         apply_negate(a[p].l);
62         apply_negate(a[p].r);
63         a[p].neg = false;
64     }
65     if (a[p].tag)
66     {
67         apply_add(a[p].l, a[p].tag);
68         apply_add(a[p].r, a[p].tag);
69         a[p].tag = 0;
70     }
71 }
72
73 // Split into the first k elements and the rest.
74 void split(int p, int k, int &l, int &r)
75 {
76     if (!p)
77     {
78         l = r = 0;
79         return;
80     }
81     push(p);
82     int left = a[a[p].l].siz;
83     if (k > left)
84     {
85         l = p;
86         split(a[p].r, k - left - 1, a[p].r, r);
87     }
88     else
89     {
90         r = p;
91         split(a[p].l, k, l, a[p].l);
92     }
93     pull(p);
94 }

```

合并、插入、删除与顺序反转（接上页同一结构体）：

```

96     int merge(int l, int r)
97     {
98         if (!l || !r) return l ? l : r;
99         if (a[l].pri > a[r].pri)
100         {
101             push(l);
102             a[l].r = merge(a[l].r, r);
103             pull(l);
104             return l;
105         }
106         push(r);
107         a[r].l = merge(l, a[r].l);
108         pull(r);
109         return r;
110     }
111
112     // Insert after the first k elements, 0 <= k <= size().
113     void insert(int k, ll value)
114     {
115         assert(0 <= k && k <= size());
116         int p = a.size(), l, r;
117         a.push_back({0, 0, 1, value, value, 0, rng(), false, false});
118         split(root, k, l, r);
119         root = merge(merge(l, p), r);
120     }
121
122     // All range operations use 1-based inclusive intervals.
123     void erase(int l, int r)
124     {
125         assert(1 <= l && l <= r && r <= size());
126         int x, y, z;
127         split(root, r, y, z);
128         split(y, l - 1, x, y);
129         root = merge(x, z);
130     }
131
132     void reverse(int l, int r)
133     {
134         assert(1 <= l && l <= r && r <= size());
135         int x, y, z;
136         split(root, r, y, z);
137         split(y, l - 1, x, y);
138         apply_reverse(y);
139         root = merge(merge(x, y), z);
140     }

```

位翻转、加法、查询与输出（接上页同一结构体）：

```

142 // Requires every value in [l,r] to be 0 or 1; preserves their order.
143 void flip_bits(int l, int r)
144 {
145     assert(1 <= l && l <= r && r <= size());
146     int x, y, z;
147     split(root, r, y, z);
148     split(y, l - 1, x, y);
149     apply_negate(y);
150     apply_add(y, 1);
151     root = merge(merge(x, y), z);
152 }
153
154 void add(int l, int r, ll value)
155 {
156     assert(1 <= l && l <= r && r <= size());
157     int x, y, z;
158     split(root, r, y, z);
159     split(y, l - 1, x, y);
160     apply_add(y, value);
161     root = merge(merge(x, y), z);
162 }
163
164 ll query(int l, int r)
165 {
166     assert(1 <= l && l <= r && r <= size());
167     int x, y, z;
168     split(root, r, y, z);
169     split(y, l - 1, x, y);
170     ll ans = a[y].sum;
171     root = merge(merge(x, y), z);
172     return ans;
173 }
174
175 void collect(int p, vector<ll> &out)
176 {
177     if (!p) return;
178     push(p);
179     collect(a[p].l, out);
180     out.push_back(a[p].val);
181     collect(a[p].r, out);
182 }
183
184 vector<ll> values()
185 {
186     vector<ll> answer;
187     collect(root, answer);
188     return answer;
189 }
190 };

```

模板题使用：Luogu P3391

题意：初始序列为 $1..n$ ，多次翻转闭区间 $[l,r]$ ，最后输出序列。插入参数为前面已有元素数，翻转区间为 1-based 闭区间。

所需模板：SequenceTreap。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P3391>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m;
6     cin >> n >> m;
7     SequenceTreap g;
8     for (int i = 1; i <= n; i++) g.insert(i - 1, i);
9     while (m--)
10     {
11         int l, r;
12         cin >> l >> r;
13         g.reverse(l, r);
14     }
15     for (long long x : g.values()) cout << x << ' ';
16     cout << '\n';
17 }
```

应用题补充：二进制序列区间取反（本地接口示例）

题意：输入 n,q 、长度 n 的 01 字符串，再输入 q 个 1-based 闭区间。每次逐位取反，最后输出字符串。保持位置顺序；要反转顺序应调用 `reverse`。按插入构造，保证随机优先级堆序；附件未提供完整牛客题号，仅登记本地应用。

所需模板：SequenceTreap。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://github.com/gsh20040816/CPCTemplate/issues/11>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, q;
6     string s;
7     cin >> n >> q >> s;
8     SequenceTreap tree;
9     for (int i = 0; i < n; i++) tree.insert(i, s[i] - '0');
10    while (q--)
11    {
12        int l, r;
13        cin >> l >> r;
14        tree.flip_bits(l, r);
15    }
16    for (auto bit : tree.values()) cout << bit;
17    cout << '\n';
18 }
```

6.2.3 状态序列 Treap：分组区间 GCD

build(v) 用数值与状态组成的序列替换当前内容。数值为无符号 64 位，状态为 0 或 1。insert(k,value,tag) 在前 k 项后插入。erase(l,r)、query(l,r,tag) 使用 1-based 非空闭区间；set(k,value) 修改值，toggle(k) 翻转状态。query 返回选中状态的 GCD，无对应元素为 nullopt，实际全零则返回 0。回收删除节点，空间按峰值长度计算。建树 $O(n)$ 次 GCD，随机优先级下单次期望 $O(\log n)$ 次 GCD；删除另需 $O(\text{删除长度})$ 回收。一次 GCD 为 $O(\log U)$ 。覆盖 kuangbin 3.7 的序列操作，尚无该原题 OJ 记录。建树后的聚合与删除节点回收均使用递归遍历；笛卡尔树的构造单调栈保留。

```

1 struct GcdSequenceTreap
2 {
3     using U = unsigned long long;
4
5     struct Node
6     {
7         int l = 0, r = 0, siz = 0, tag = 0;
8         int cnt[2]{};
9         U val = 0, g[2]{}, pri = 0;
10    };
11
12    vector<Node> a{Node{}};
13    vector<int> recycled;
14    mt19937_64 rng;
15    int root = 0;
16
17    GcdSequenceTreap(U seed = 712367821) : rng(seed) {}
18
19    int size() const { return a[root].siz; }
20
21    int node(U value, int tag)
22    {
23        assert(tag == 0 || tag == 1);
24        int p;
25        if (recycled.empty())
26        {
27            p = a.size();
28            a.push_back(Node{});
29        }
30        else
31        {
32            p = recycled.back();
33            recycled.pop_back();
34            a[p] = Node{};
35        }
36        a[p].siz = 1;
37        a[p].val = value;
38        a[p].tag = tag;
39        a[p].cnt[tag] = 1;
40        a[p].g[tag] = value;
41        a[p].pri = rng();
42        return p;
43    }
44
45    void pull(int p)
46    {
47        int l = a[p].l, r = a[p].r;

```

```

48     a[p].siz = a[l].siz + a[r].siz + 1;
49     for (int t = 0; t < 2; t++)
50     {
51         a[p].cnt[t] = a[l].cnt[t] + a[r].cnt[t] + (a[p].tag == t);
52         a[p].g[t] = gcd(a[l].g[t], a[r].g[t]);
53         if (a[p].tag == t) a[p].g[t] = gcd(a[p].g[t], a[p].val);
54     }
55 }
56
57 void split(int p, int k, int &l, int &r)
58 {
59     if (!p)
60     {
61         l = 0;
62         r = 0;
63         return;
64     }
65     int left = a[a[p].l].siz;
66     if (k > left)
67     {
68         l = p;
69         split(a[p].r, k - left - 1, a[p].r, r);
70     }
71     else
72     {
73         r = p;
74         split(a[p].l, k, l, a[p].l);
75     }
76     pull(p);
77 }
78
79 int merge(int l, int r)
80 {
81     if (!l || !r) return l ? l : r;
82     if (a[l].pri > a[r].pri)
83     {
84         a[l].r = merge(a[l].r, r);
85         pull(l);
86         return l;
87     }
88     a[r].l = merge(l, a[r].l);
89     pull(r);
90     return r;
91 }
92
93 void rebuild(int p)
94 {
95     if (!p) return;
96     rebuild(a[p].l);
97     rebuild(a[p].r);
98     pull(p);
99 }
100
101 void release(int p)
102 {
103     if (!p) return;

```

```

104     release(a[p].l);
105     release(a[p].r);
106     recycled.push_back(p);
107 }
108
109 // O(n) GCD calls; replaces the current sequence.
110 void build(const vector<pair<U, int>> &v)
111 {
112     a.assign(1, Node{});
113     recycled.clear();
114     root = 0;
115     vector<int> st;
116     for (auto [value, tag] : v)
117     {
118         int p = node(value, tag), last = 0;
119         while (!st.empty() && a[st.back()].pri < a[p].pri)
120         {
121             last = st.back();
122             st.pop_back();
123         }
124         a[p].l = last;
125         if (!st.empty()) a[st.back()].r = p;
126         st.push_back(p);
127     }
128     if (st.empty()) return;
129     root = st.front();
130     rebuild(root);
131 }
132
133 // Insert after the first k entries. Range operations below are 1-based.
134 void insert(int k, U value, int tag)
135 {
136     assert(0 <= k && k <= size());
137     int p = node(value, tag), l, r;
138     split(root, k, l, r);
139     root = merge(merge(l, p), r);
140 }
141
142 void erase(int l, int r)
143 {
144     assert(1 <= l && l <= r && r <= size());
145     int x, y, z;
146     split(root, r, y, z);
147     split(y, l - 1, x, y);
148     root = merge(x, z);
149     release(y);
150 }
151
152 void set(int k, U value)
153 {
154     assert(1 <= k && k <= size());
155     int x, y, z;
156     split(root, k, y, z);
157     split(y, k - 1, x, y);
158     a[y].val = value;
159     pull(y);

```

```

160     root = merge(merge(x, y), z);
161 }
162
163 void toggle(int k)
164 {
165     assert(1 <= k && k <= size());
166     int x, y, z;
167     split(root, k, y, z);
168     split(y, k - 1, x, y);
169     a[y].tag ^= 1;
170     pull(y);
171     root = merge(merge(x, y), z);
172 }
173
174 optional<U> query(int l, int r, int tag)
175 {
176     assert(1 <= l && l <= r && r <= size());
177     assert(tag == 0 || tag == 1);
178     int x, y, z;
179     split(root, r, y, z);
180     split(y, l - 1, x, y);
181     optional<U> answer;
182     if (a[y].cnt[tag]) answer = a[y].g[tag];
183     root = merge(merge(x, y), z);
184     return answer;
185 }
186 };

```

6.3 Splay 树

6.3.1 Splay：可重集合与顺序统计

insert/erase/rank/kth/prev/next 与有序 Treap 同义，严格前驱后继不存在返回 nullopt。重复键计数存储；失败查询也伸展最后访问节点，单次均摊 $O(\log n)$ 。不回收节点，空间按累计新建节点数计算。支持 long long 全范围；不包含序列 Splay、虚子树信息或矩阵动态 DP。

```

1 struct OrderedSplay
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int ch[2]{}, fa = 0, cnt = 0, siz = 0;
8         ll val = 0;
9     };
10
11     vector<Node> a{Node{}};
12     int root = 0;
13
14     int size() const { return a[root].siz; }
15
16     void pull(int x) { a[x].siz = a[a[x].ch[0]].siz + a[x].cnt + a[a[x].ch[1]].siz; }
17
18     int node(ll value, int parent)
19     {

```

```

20     int x = a.size();
21     a.push_back({0, 0}, parent, 1, 1, value});
22     return x;
23 }
24
25 void rotate(int x)
26 {
27     int y = a[x].fa, z = a[y].fa;
28     int k = a[y].ch[1] == x;
29     int w = a[x].ch[k ^ 1];
30     if (z)
31         a[z].ch[a[z].ch[1] == y] = x;
32     else
33         root = x;
34     a[x].fa = z;
35     a[y].ch[k] = w;
36     if (w) a[w].fa = y;
37     a[x].ch[k ^ 1] = y;
38     a[y].fa = x;
39     pull(y);
40     pull(x);
41 }
42
43 void splay(int x)
44 {
45     while (a[x].fa)
46     {
47         int y = a[x].fa, z = a[y].fa;
48         if (z)
49         {
50             bool same = (a[y].ch[1] == x) == (a[z].ch[1] == y);
51             rotate(same ? y : x);
52         }
53         rotate(x);
54     }
55 }
56
57 // Splay the last visited node even when the value does not exist.
58 int find(ll value)
59 {
60     int x = root, last = 0;
61     while (x && a[x].val != value)
62     {
63         last = x;
64         x = a[x].ch[value > a[x].val];
65     }
66     if (x)
67         splay(x);
68     else if (last)
69         splay(last);
70     return x;
71 }
72
73 void insert(ll value)
74 {
75     int x = root, parent = 0;

```

```

76     while (x && a[x].val != value)
77     {
78         parent = x;
79         x = a[x].ch[value > a[x].val];
80     }
81     if (x)
82     {
83         ++a[x].cnt;
84         pull(x);
85     }
86     else
87     {
88         x = node(value, parent);
89         if (parent)
90             a[parent].ch[value > a[parent].val] = x;
91         else
92             root = x;
93     }
94     splay(x);
95 }
96
97 bool erase(ll value)
98 {
99     int x = find(value);
100    if (!x) return false;
101    if (a[x].cnt > 1)
102    {
103        --a[x].cnt;
104        pull(x);
105        return true;
106    }
107    int l = a[x].ch[0], r = a[x].ch[1];
108    if (l) a[l].fa = 0;
109    if (r) a[r].fa = 0;
110    if (!l)
111    {
112        root = r;
113        return true;
114    }
115    root = l;
116    x = l;
117    while (a[x].ch[1]) x = a[x].ch[1];
118    splay(x);
119    a[x].ch[1] = r;
120    if (r) a[r].fa = x;
121    pull(x);
122    return true;
123 }
124
125 int rank(ll value)
126 {
127     int x = root, last = 0, ans = 1;
128     while (x)
129     {
130         last = x;
131         if (value <= a[x].val)

```

```

132         x = a[x].ch[0];
133     else
134     {
135         ans += a[a[x].ch[0]].siz + a[x].cnt;
136         x = a[x].ch[1];
137     }
138 }
139 if (last) splay(last);
140 return ans;
141 }
142
143 ll kth(int k)
144 {
145     assert(1 <= k && k <= size());
146     int x = root;
147     while (true)
148     {
149         int left = a[a[x].ch[0]].siz;
150         if (k <= left)
151             x = a[x].ch[0];
152         else if (k <= left + a[x].cnt)
153         {
154             splay(x);
155             return a[x].val;
156         }
157         else
158         {
159             k -= left + a[x].cnt;
160             x = a[x].ch[1];
161         }
162     }
163 }
164
165 optional<ll> prev(ll value)
166 {
167     int x = root, last = 0, answer = 0;
168     while (x)
169     {
170         last = x;
171         if (a[x].val < value)
172         {
173             answer = x;
174             x = a[x].ch[1];
175         }
176         else
177             x = a[x].ch[0];
178     }
179     if (last) splay(last);
180     if (!answer) return nullopt;
181     splay(answer);
182     return a[answer].val;
183 }
184
185 optional<ll> next(ll value)
186 {
187     int x = root, last = 0, answer = 0;

```

```

188     while (x)
189     {
190         last = x;
191         if (a[x].val > value)
192         {
193             answer = x;
194             x = a[x].ch[0];
195         }
196         else
197             x = a[x].ch[1];
198     }
199     if (last) splay(last);
200     if (!answer) return nullopt;
201     splay(answer);
202     return a[answer].val;
203 }
204 };

```

模板题使用：Luogu P3369

题意：维护可重复整数集合，支持插入、删除一个值、排名、第 k 小、严格前驱和严格后继。题目保证前驱/后继存在，因此可解引用返回值。

所需模板：OrderedSplay。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P3369>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n;
6     cin >> n;
7     OrderedSplay g;
8     while (n--)
9     {
10         int op;
11         long long x;
12         cin >> op >> x;
13         if (op == 1)
14             g.insert(x);
15         else if (op == 2)
16             g.erase(x);
17         else if (op == 3)
18             cout << g.rank(x) << '\n';
19         else if (op == 4)
20             cout << g.kth(x) << '\n';
21         else if (op == 5)
22             cout << *g.prev(x) << '\n';
23         else
24             cout << *g.next(x) << '\n';
25     }
26 }

```

6.4 替罪羊树

6.4.1 替罪羊树：可重集合与顺序统计

默认空树；insert(x) 插入一个副本，erase(x) 删除一个副本并返回是否存在。less(x) 计严格小于 x 的元素，less(x,true) 计小于等于；rank(x)=less(x)+1, x 不必存在。kth(k) 要求 $1 \leq k \leq \text{size}()$ 。prev/next 返回严格前驱/后继，不存在为 nullopt。键支持完整 long long，不使用数值哨兵。节点数过度偏向单侧或有效不同值不足一半时递归重建，复用活节点编号。若 d 为有效不同值数，查询 $O(\log(d+2))$ ，更新摊 $O(\log(d+2))$ 、单次最坏 $O(d+1)$ 。元素数与累计分配节点数均须小于 INT_MAX。内存 $O(K)$ ，K 为累计分配节点数；墓碑从树中清除后不回收到节点池，重建本身不增分配量。a、root 和辅助接口只供内部使用；重新赋值 ScapegoatTree() 可清空。

```

1 struct ScapegoatTree
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int l = 0, r = 0, count = 0, size = 0, nodes = 0, live = 0;
8         ll value = 0;
9     };
10
11     vector<Node> a{Node{}};
12     int root = 0;
13
14     int size() const { return a[root].size; }
15
16     void pull(int p)
17     {
18         int l = a[p].l, r = a[p].r;
19         a[p].size = a[l].size + a[r].size + a[p].count;
20         a[p].nodes = a[l].nodes + a[r].nodes + 1;
21         a[p].live = a[l].live + a[r].live + (a[p].count > 0);
22     }
23
24     void collect(int p, vector<int> &v)
25     {
26         if (!p) return;
27         collect(a[p].l, v);
28         if (a[p].count) v.push_back(p);
29         collect(a[p].r, v);
30     }
31
32     int build(const vector<int> &v, int l, int r)
33     {
34         if (l == r) return 0;
35         int m = l + (r - l) / 2, p = v[m];
36         a[p].l = build(v, l, m);
37         a[p].r = build(v, m + 1, r);
38         pull(p);
39         return p;
40     }

```

重建与插入删除（接上页同一结构体）：

```

42     int balance(int p)
43     {
44         pull(p);
45         int heavy = max(a[a[p].l].nodes, a[a[p].r].nodes);
46         if (4LL * heavy <= 3LL * a[p].nodes && 2LL * a[p].live >= a[p].nodes) return p;
47         vector<int> v;
48         v.reserve(a[p].live);
49         collect(p, v);
50         return build(v, 0, v.size());
51     }
52
53     int insert(int p, ll x)
54     {
55         if (!p)
56         {
57             a.push_back({0, 0, 1, 1, 1, 1, x});
58             return a.size() - 1;
59         }
60         if (x < a[p].value)
61             a[p].l = insert(a[p].l, x);
62         else if (x > a[p].value)
63             a[p].r = insert(a[p].r, x);
64         else
65             a[p].count++;
66         return balance(p);
67     }
68
69     int erase(int p, ll x, bool &found)
70     {
71         if (!p) return 0;
72         if (x < a[p].value)
73             a[p].l = erase(a[p].l, x, found);
74         else if (x > a[p].value)
75             a[p].r = erase(a[p].r, x, found);
76         else if (a[p].count)
77         {
78             a[p].count--;
79             found = true;
80         }
81         return balance(p);
82     }
83
84     void insert(ll x) { root = insert(root, x); }
85
86     bool erase(ll x)
87     {
88         bool found = false;
89         root = erase(root, x, found);
90         return found;
91     }

```

排名、第 k 小与严格前驱后继（接上页同一结构体）：

```

93     int less(ll x, bool equal = false) const
94     {
95         int p = root, answer = 0;
96         while (p)
97         {
98             if (a[p].value < x || (equal && a[p].value == x))
99             {
100                 answer += a[a[p].l].size + a[p].count;
101                 p = a[p].r;
102             }
103             else
104                 p = a[p].l;
105         }
106         return answer;
107     }
108
109     int rank(ll x) const { return less(x) + 1; }
110
111     ll kth(int k) const
112     {
113         assert(1 <= k && k <= size());
114         int p = root;
115         while (true)
116         {
117             int left = a[a[p].l].size;
118             if (k <= left)
119                 p = a[p].l;
120             else if (k <= left + a[p].count)
121                 return a[p].value;
122             else
123             {
124                 k -= left + a[p].count;
125                 p = a[p].r;
126             }
127         }
128     }
129
130     optional<ll> prev(ll x) const
131     {
132         int k = less(x);
133         if (!k) return nullopt;
134         return kth(k);
135     }
136
137     optional<ll> next(ll x) const
138     {
139         int k = less(x, true);
140         if (k == size()) return nullopt;
141         return kth(k + 1);
142     }
143 };

```

模板题使用：Luogu P3369 (替罪羊树)

题意：维护可重集合，操作 1..6 分别为插入、删除一个副本、排名、第 k 小、严格前驱、严格后继。排名查询键不一定存在；题目保证第 k 小和前驱后继的答案存在，因此可解引用 optional。该驱动单独调用替罪羊树，不沿用其他平衡树的 AC。

所需模板：ScapegoatTree。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P3369>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n;
6     cin >> n;
7     ScapegoatTree g;
8     while (n--)
9     {
10         int op;
11         long long x;
12         cin >> op >> x;
13         if (op == 1)
14             g.insert(x);
15         else if (op == 2)
16             g.erase(x);
17         else if (op == 3)
18             cout << g.rank(x) << '\n';
19         else if (op == 4)
20             cout << g.kth(x) << '\n';
21         else if (op == 5)
22             cout << *g.prev(x) << '\n';
23         else
24             cout << *g.next(x) << '\n';
25     }
26 }
```

第七章 可持久化数据结构

7.1 可持久化线段树

7.1.1 主席树：静态区间第 k 小

构造传入 0-based 数组， $kth(l,r,k)$ 的区间使用 1-based 闭区间；要求 $1 \leq k \leq r-l+1$ 。离散化后每个前缀一个版本，单次 $O(\log n)$ ，总空间 $O(n \log n)$ 。

```
1 struct PersistentKth
2 {
3     struct Node
4     {
5         int l = 0, r = 0, sum = 0;
6     };
7
8     vector<Node> t{Node{}};
9     vector<int> root{0};
10    vector<long long> vals;
11
12    PersistentKth(const vector<long long> &a) : vals(a)
13    {
14        sort(vals.begin(), vals.end());
15        vals.erase(unique(vals.begin(), vals.end()), vals.end());
16        for (long long x : a)
17        {
18            int p = lower_bound(vals.begin(), vals.end(), x) - vals.begin();
19            root.push_back(add(root.back(), 0, (int)vals.size() - 1, p));
20        }
21    }
22
23    int add(int old, int l, int r, int x)
24    {
25        int p = t.size();
26        Node copy = t[old];
27        t.push_back(copy);
28        ++t[p].sum;
29        if (l < r)
30        {
31            int m = (l + r) / 2;
32            if (x <= m)
33            {
34                int v = add(t[old].l, l, m, x);
35                t[p].l = v;
36            }
37            else
38            {
39                int v = add(t[old].r, m + 1, r, x);
40                t[p].r = v;
41            }
42        }
43        return p;
44    }
```

```

44     }
45
46     long long kth(int l, int r, int k) const
47     {
48         assert(1 <= l && l <= r && r < (int)root.size() && 1 <= k && k <= r - l + 1);
49         int u = root[l - 1], v = root[r], L = 0, R = (int)vals.size() - 1;
50         while (L < R)
51         {
52             int m = (L + R) / 2, c = t[t[v].l].sum - t[t[u].l].sum;
53             if (k <= c)
54             {
55                 u = t[u].l;
56                 v = t[v].l;
57                 R = m;
58             }
59             else
60             {
61                 k -= c;
62                 u = t[u].r;
63                 v = t[v].r;
64                 L = m + 1;
65             }
66         }
67         return vals[L];
68     }
69 };

```

模板题使用：Luogu P3834

题意：给定不再修改的数组，查询闭区间 $[l, r]$ 内第 k 小的值，重复值按出现次数计数。构造传入 0-based vector，查询 l 、 r 、 k 均从 1 开始。

所需模板：PersistentKth。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P3834>

```

1  int main()
2  {
3      std::ios::sync_with_stdio(false);
4      std::cin.tie(nullptr);
5      int n, m;
6      std::cin >> n >> m;
7      std::vector<long long> a(n);
8      for (auto &x : a) std::cin >> x;
9      PersistentKth tree(a);
10     while (m--)
11     {
12         int l, r, k;
13         std::cin >> l >> r >> k;
14         std::cout << tree.kth(l, r, k) << '\n';
15     }
16 }

```

7.1.2 可持久化数组：历史版本分叉与单点修改

PersistentArray(a,changes=0), a 是非空 0-based 初值数组；第 0 版为初始状态。set(version,pos,value) 基于任意已有版本修改 1-based 位置，创建并返回新版本编号；copy(version) 以 $O(1)$ 时间复制版本；query(version,pos) 只读且不自动创建版本。values 支持 long long 全范围。每次修改复制根到叶的节点，旧版本不变；build 为内部函数，不可覆盖历史节点。建树 $O(n)$ ，修改及查询 $O(\log n)$ ，初始 $2n-1$ 个节点，每次修改至多 $\text{ceil}(\log_2(n))+1$ 个；总空间 $O(n + \text{修改数} * \log n + \text{版本数})$ 。changes 仅用于预留节点池，低估后可扩容，所有外部引用应使用下标，不保存 vector 元素引用。节点下标用 int，节点总量须可表示。

```

1 struct PersistentArray
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int l = 0, r = 0;
8         ll value = 0;
9     };
10
11     int n;
12     vector<Node> t{Node{}};
13     vector<int> root;
14
15     PersistentArray(const vector<ll> &a, int changes = 0) : n(a.size())
16     {
17         assert(n > 0 && changes >= 0);
18         int levels = 1;
19         for (int x = n - 1; x; x >>= 1) levels++;
20         t.reserve(2ULL * n + 1ULL * changes * levels + 1);
21         root.push_back(build(1, n, a));
22     }
23
24     int build(int l, int r, const vector<ll> &a)
25     {
26         int p = t.size();
27         t.push_back(Node{});
28         if (l == r)
29             t[p].value = a[l - 1];
30         else
31         {
32             int m = (l + r) / 2;
33             int left = build(l, m, a);
34             int right = build(m + 1, r, a);
35             t[p].l = left;
36             t[p].r = right;
37         }
38         return p;
39     }
40
41     int modify(int old, int l, int r, int pos, ll value)
42     {
43         int p = t.size();
44         Node copy = t[old];
45         t.push_back(copy);
46         if (l == r)
47             t[p].value = value;

```

```

48     else
49     {
50         int m = (l + r) / 2;
51         if (pos <= m)
52         {
53             int child = modify(t[old].l, l, m, pos, value);
54             t[p].l = child;
55         }
56         else
57         {
58             int child = modify(t[old].r, m + 1, r, pos, value);
59             t[p].r = child;
60         }
61     }
62     return p;
63 }
64
65 // Create a new version from any existing version. Positions are 1-based.
66 int set(int version, int pos, ll value)
67 {
68     assert(0 <= version && version < (int)root.size());
69     assert(1 <= pos && pos <= n);
70     int p = modify(root[version], 1, n, pos, value);
71     root.push_back(p);
72     return (int)root.size() - 1;
73 }
74
75 int copy(int version)
76 {
77     assert(0 <= version && version < (int)root.size());
78     root.push_back(root[version]);
79     return (int)root.size() - 1;
80 }
81
82 ll query(int version, int pos) const
83 {
84     assert(0 <= version && version < (int)root.size());
85     assert(1 <= pos && pos <= n);
86     int p = root[version], l = 1, r = n;
87     while (l < r)
88     {
89         int m = (l + r) / 2;
90         if (pos <= m)
91         {
92             p = t[p].l;
93             r = m;

```

```

94         }
95         else
96         {
97             p = t[p].r;
98             l = m + 1;
99         }
100     }
101     return t[p].value;
102 }
103 };

```

模板题使用：Luogu P3919

题意：从指定历史版本修改单点或查询单点，每次操作都创建新版本。查询后也要 copy 原版本；版本号从 0 开始，位置为 1-based。

所需模板：PersistentArray。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P3919>

```

1 int main()
2 {
3     std::ios::sync_with_stdio(false);
4     std::cin.tie(nullptr);
5     int n, m;
6     std::cin >> n >> m;
7     std::vector<long long> a(n);
8     for (auto &x : a) std::cin >> x;
9     PersistentArray tree(a, m);
10    while (m--)
11    {
12        int version, op, pos;
13        std::cin >> version >> op >> pos;
14        if (op == 1)
15        {
16            long long value;
17            std::cin >> value;
18            tree.set(version, pos, value);
19        }
20        else
21        {
22            std::cout << tree.query(version, pos) << '\n';
23            tree.copy(version);
24        }
25    }
26 }

```

7.1.3 可持久化区间加与版本拼接

PersistentRange(n) 建立长度 n 的全零版本 0, $n \geq 1$ ；使用隐式节点，不预建整棵树。add(v,l,r,x) 在版本 v 的 [l,r) 加 x；splice(dst,src,l,r) 将 dst 的 [l,r) 替换为 src 同一位置的值，其他位置保留 dst；二者均返回新版本编号。query(v,l,r) 返回区间和，const 且不分配节点。下标 0-based、区间半开，允许空区间；空修改及零增量仍创建版本但复用根，整段拼接直接共享源根。全体版本属于同一对象、同一坐标域。懒标记保留在祖先，用 offset 修正跨版本拼接；每次修改/拼接/查询 $O(\log n)$ ，版本别名 $O(1)$ ，总节点 $O(q \log n)$ 。sum、lazy、delta、carry、offset 及其中间和差和长度乘积均须在 long long 内；节点总数和版本数须小于 INT_MAX。t、root 可读以统计空间或查看根，不要外部改写。

```
1 struct PersistentRange
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int left = 0, right = 0;
8         ll sum = 0, lazy = 0;
9     };
10
11     int n;
12     vector<Node> t{Node{}};
13     vector<int> root{0};
14
15     explicit PersistentRange(int size) : n(size) { assert(n > 0); }
16
17     int clone(int p)
18     {
19         Node node = t[p];
20         t.push_back(node);
21         return int(t.size()) - 1;
22     }
23
24     void pull(int p, int len)
25     {
26         t[p].sum = t[t[p].left].sum + t[t[p].right].sum + t[p].lazy * len;
27     }
```

区间加与版本拼接（接上页同一结构体）：

```

29     int change(int p, int l, int r, int ql, int qr, ll value)
30     {
31         if (qr <= l || r <= ql) return p;
32         int u = clone(p);
33         if (ql <= l && r <= qr)
34         {
35             t[u].lazy += value;
36             t[u].sum += value * (r - l);
37             return u;
38         }
39         int m = l + (r - l) / 2;
40         int a = change(t[p].left, l, m, ql, qr, value);
41         int b = change(t[p].right, m, r, ql, qr, value);
42         t[u].left = a;
43         t[u].right = b;
44         pull(u, r - l);
45         return u;
46     }
47
48     int replace(int p, int q, int l, int r, int ql, int qr, ll offset)
49     {
50         if (qr <= l || r <= ql) return p;
51         if (ql <= l && r <= qr)
52         {
53             if (offset == 0) return q;
54             int u = clone(q);
55             t[u].lazy += offset;
56             t[u].sum += offset * (r - l);
57             return u;
58         }
59         int u = clone(p), m = l + (r - l) / 2;
60         offset += t[q].lazy - t[p].lazy;
61         int a = replace(t[p].left, t[q].left, l, m, ql, qr, offset);
62         int b = replace(t[p].right, t[q].right, m, r, ql, qr, offset);
63         t[u].left = a;
64         t[u].right = b;
65         pull(u, r - l);
66         return u;
67     }

```

只读查询与对外接口（接上页同一结构体）：

```

69 ll get(int p, int l, int r, int ql, int qr, ll carry) const
70 {
71     if (qr <= l || r <= ql) return 0;
72     if (ql <= l && r <= qr) return t[p].sum + carry * (r - l);
73     carry += t[p].lazy;
74     int m = l + (r - l) / 2;
75     return get(t[p].left, l, m, ql, qr, carry) +
76            get(t[p].right, m, r, ql, qr, carry);
77 }
78
79 void check(int version, int l, int r) const
80 {
81     assert(0 <= version && version < int(root.size()));
82     assert(0 <= l && l <= r && r <= n);
83 }
84
85 int add(int version, int l, int r, ll value)
86 {
87     check(version, l, r);
88     int p = l == r || value == 0 ? root[version]
89           : change(root[version], 0, n, l, r, value);
90     root.push_back(p);
91     return int(root.size()) - 1;
92 }
93
94 int splice(int destination, int source, int l, int r)
95 {
96     check(destination, l, r);
97     check(source, l, r);
98     int p = l == r ? root[destination]
99           : replace(root[destination], root[source], 0, n, l, r, 0);
100    root.push_back(p);
101    return int(root.size()) - 1;
102 }
103
104 ll query(int version, int l, int r) const
105 {
106     check(version, l, r);
107     return l == r ? 0 : get(root[version], 0, n, l, r, 0);
108 }
109 };

```

7.1.4 树上路径点权第 k 小

TreePathKth(a) 传入非空 0-based 权值数组，节点编号 1..n；add(u,v) 加无向边，要求恰好构成一棵树。build(root=1) 递归 DFS 建立根到各节点的频数版本与倍增表，可换根重建，旧版本自动清空。kth(u,v,k) 返回简单路径上第 k 小的点权，包含两个端点，每个节点计一次； $1 \leq k \leq$ 路径点数。lca(u,v) 返回当前预处理根下的最近公共祖先。使用 $\text{root}[u] + \text{root}[v] - \text{root}[\text{lca}] - \text{root}[\text{parent}(\text{lca})]$ ，编号 0 为全零版本，避免少算或重复算 LCA。支持重复值与完整 long long 权值，内部只统计频数；权值和树结构在查询阶段保持不变。预处理 $O(n \log n)$ ，每次查询 $O(\log n)$ ，空间 $O(n \log n)$ 。递归深度最坏 $O(n)$ ，保持递归写法。

```

1 struct TreePathKth
2 {

```

```

3      using ll = long long;
4
5      struct Node
6      {
7          int l = 0, r = 0, sum = 0;
8      };
9
10     int n, lg;
11     vector<vector<int>> g, up;
12     vector<int> root, dep;
13     vector<ll> value, vals;
14     vector<Node> t;
15
16     TreePathKth(const vector<ll> &a)
17         : n(a.size()), g(n + 1), root(n + 1), dep(n + 1), value(a)
18     {
19         assert(n > 0);
20         lg = 1;
21         while ((1LL << lg) <= n) lg++;
22         up.assign(lg, vector<int>(n + 1));
23     }
24
25     void add(int u, int v)
26     {
27         g[u].push_back(v);
28         g[v].push_back(u);
29     }
30
31     int insert(int old, int l, int r, int x)
32     {
33         int p = t.size();
34         Node copy = t[old];
35         t.push_back(copy);
36         t[p].sum++;
37         if (l == r) return p;
38         int m = (l + r) / 2;
39         if (x <= m)
40         {
41             int child = insert(t[old].l, l, m, x);
42             t[p].l = child;
43         }
44         else
45         {
46             int child = insert(t[old].r, m + 1, r, x);
47             t[p].r = child;
48         }
49         return p;
50     }
51
52     void dfs(int u, int p)
53     {
54         up[0][u] = p;
55         for (int j = 1; j < lg; j++) up[j][u] = up[j - 1][up[j - 1][u]];
56         int x = lower_bound(vals.begin(), vals.end(), value[u - 1]) - vals.begin();
57         root[u] = insert(root[p], 0, (int)vals.size() - 1, x);
58         for (int v : g[u])

```

```

59     {
60         if (v == p) continue;
61         dep[v] = dep[u] + 1;
62         dfs(v, u);
63     }
64 }
65
66 void build(int r = 1)
67 {
68     assert(1 <= r && r <= n);
69     vals = value;
70     sort(vals.begin(), vals.end());
71     vals.erase(unique(vals.begin(), vals.end()), vals.end());
72     t.assign(1, Node{});
73     dep[r] = 0;
74     dfs(r, 0);
75 }
76
77 int lca(int u, int v) const
78 {
79     if (dep[u] < dep[v]) swap(u, v);
80     int diff = dep[u] - dep[v];
81     for (int j = 0; j < lg; j++)
82         if (diff >> j & 1) u = up[j][u];
83     if (u == v) return u;
84     for (int j = lg - 1; j >= 0; j--)
85         if (up[j][u] != up[j][v])
86             {
87                 u = up[j][u];
88                 v = up[j][v];
89             }
90     return up[0][u];
91 }
92
93 ll kth(int u, int v, int k) const
94 {
95     int z = lca(u, v);
96     assert(1 <= k && k <= dep[u] + dep[v] - 2 * dep[z] + 1);
97     int a = root[u], b = root[v], c = root[z], d = root[up[0][z]];
98     int l = 0, r = (int)vals.size() - 1;
99     while (l < r)
100     {
101         int count = t[t[a].l].sum + t[t[b].l].sum - t[t[c].l].sum - t[t[d].l].sum;
102         int m = (l + r) / 2;
103         if (k <= count)
104             {
105                 a = t[a].l;
106                 b = t[b].l;
107                 c = t[c].l;
108                 d = t[d].l;
109                 r = m;
110             }
111         else
112             {
113                 k -= count;

```

```

114         a = t[a].r;
115         b = t[b].r;
116         c = t[c].r;
117         d = t[d].r;
118         l = m + 1;
119     }
120 }
121 return vals[l];
122 }
123 };

```

模板题使用：Luogu P2633

题意：给点权树，查询路径上第 k 小的点权，包含两个端点。构造传入 0-based 权值数组，点号为 $1..n$ ；每次先将输入的的第一个端点与上次答案异或解码，初始 $last=0$ 。

所需模板：TreePathKth。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P2633>

```

1 int main()
2 {
3     std::ios::sync_with_stdio(false);
4     std::cin.tie(nullptr);
5     int n, m;
6     std::cin >> n >> m;
7     std::vector<long long> a(n);
8     for (auto &x : a) std::cin >> x;
9     TreePathKth tree(a);
10    for (int i = 1; i < n; i++)
11    {
12        int u, v;
13        std::cin >> u >> v;
14        tree.add(u, v);
15    }
16    tree.build();
17    long long last = 0;
18    while (m--)
19    {
20        long long encoded;
21        int v, k;
22        std::cin >> encoded >> v >> k;
23        int u = encoded ^ last;
24        last = tree.kth(u, v, k);
25        std::cout << last << '\n';
26    }
27 }

```

7.1.5 静态区间不同数计数（前驱位置主席树）

PersistentDistinct(a) 传入非空 0-based 数组， $query(l, r)$ 返回闭区间内不同值的个数， $1 \leq l \leq r \leq n$ 。支持重复值与完整 `long long` 值域；建成后不支持改值，查询顺序任意。令 $prev[i]$ 为同值上次出现的位置，未出现为 0； $root[i]$ 在值域 $0..n-1$ 中统计前 i 个 $prev$ 。区间内每种值恰有一次 $prev[i] < l$ ，因此答案为 $root[r]-root[l-1]$ 中位置 $0..l-1$ 的频数和。每个元素只复制一条节点路径，预处理 $O(n \log n)$ ，查询 $O(\log n)$ ，空间 $O(n \log n)$ 。动态版预留 $n \cdot (\text{ceil}(\log_2(n))+1)+1$ 个节点，避免扩容峰值； $root$ 和节点池为内部数据。

```

1 struct PersistentDistinct
2 {
3     struct Node
4     {
5         int l = 0, r = 0, sum = 0;
6     };
7
8     int n;
9     vector<Node> t{Node{}};
10    vector<int> root;
11
12    PersistentDistinct(const vector<long long> &a) : n(a.size()), root(n + 1)
13    {
14        assert(n > 0);
15        int height = 1;
16        while ((1LL << (height - 1)) < n) height++;
17        t.reserve(size_t(n) * height + 1);
18        vector<long long> vals = a;
19        sort(vals.begin(), vals.end());
20        vals.erase(unique(vals.begin(), vals.end()), vals.end());
21        vector<int> last(vals.size());
22        for (int i = 1; i <= n; i++)
23        {
24            int x = lower_bound(vals.begin(), vals.end(), a[i - 1]) - vals.begin();
25            root[i] = insert(root[i - 1], 0, n - 1, last[x]);
26            last[x] = i;
27        }
28    }
29
30    int insert(int old, int l, int r, int x)
31    {
32        int p = t.size();
33        Node copy = t[old];
34        t.push_back(copy);
35        t[p].sum++;
36        if (l < r)
37        {
38            int m = (l + r) / 2;
39            if (x <= m)
40            {
41                int child = insert(t[old].l, l, m, x);
42                t[p].l = child;
43            }
44            else
45            {
46                int child = insert(t[old].r, m + 1, r, x);
47                t[p].r = child;
48            }
49        }
50        return p;
51    }
52
53    int query(int l, int r) const
54    {
55        assert(1 <= l && l <= r && r <= n);

```

```

56     int u = root[l - 1], v = root[r];
57     int L = 0, R = n - 1, answer = 0;
58     while (L < R)
59     {
60         int m = (L + R) / 2;
61         if (l - 1 <= m)
62         {
63             u = t[u].l;
64             v = t[v].l;
65             R = m;
66         }
67         else
68         {
69             answer += t[t[v].l].sum - t[t[u].l].sum;
70             u = t[u].r;
71             v = t[v].r;
72             L = m + 1;
73         }
74     }
75     return answer + t[v].sum - t[u].sum;
76 }
77 };

```

应用题补充：Luogu P1972

题意：给定固定数组，查询闭区间 $[l, r]$ 内不同值的个数。构造传入 0-based vector，查询使用 1-based 闭区间；重复出现的同一值只计一次。

所需模板：PersistentDistinct。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P1972>

```

1  int main()
2  {
3      std::ios::sync_with_stdio(false);
4      std::cin.tie(nullptr);
5      int n;
6      std::cin >> n;
7      std::vector<long long> a(n);
8      for (auto &x : a) std::cin >> x;
9      PersistentDistinct tree(a);
10     int m;
11     std::cin >> m;
12     while (m--)
13     {
14         int l, r;
15         std::cin >> l >> r;
16         std::cout << tree.query(l, r) << '\n';
17     }
18 }

```

第八章 树套树

8.1 树状数组套权值线段树

8.1.1 动态区间第 k 小（树状数组套权值线段树）

DynamicKth(a,future) 传入非空 0-based 初值, future 包含所有将来可能赋入的新值; 内部自动合并初值并离散化。set(pos,value) 单点赋值, kth(l,r,k) 查询闭区间第 k 小, 下标均为 1-based; $1 \leq k \leq r-l+1$ 。允许重复值与完整 long long 权值; 新值必须已在离散化集合内, 操作可按原顺序执行, 不保留历史版本。每个树状数组节点对应一棵可变权值线段树; 删除旧值后插入新值。空节点回收到 free_head 链, 重复修改不会按操作次数累计泄漏节点。设 D 为不同值数, $S=n+\text{future.size()}$, 预处理 $O(S \log S + n \log n \log(D+1))$, 单次修改及查询 $O(\log n \log(D+1))$ 。节点数上界 $B=(\text{ceil}(\log_2(D))+1) \cdot \sum \text{lowbit}(i)$, 求和 $i=1..n$; 空间 $O(S+n+B)$ 。update、modify、id 和节点池均为内部接口; 不能手动减去不存在的值。

```
1 struct DynamicKth
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int l = 0, r = 0, sum = 0;
8     };
9
10    int n, free_head = 0;
11    vector<int> bit, rank;
12    vector<ll> vals;
13    vector<Node> t{Node{}};
14
15    DynamicKth(const vector<ll> &a, vector<ll> future = {})
16        : n(a.size()), bit(n + 1), rank(n + 1)
17    {
18        assert(n > 0);
19        vals = future;
20        vals.insert(vals.end(), a.begin(), a.end());
21        sort(vals.begin(), vals.end());
22        vals.erase(unique(vals.begin(), vals.end()), vals.end());
23        for (int i = 1; i <= n; i++)
24        {
25            rank[i] = id(a[i - 1]);
26            modify(i, rank[i], 1);
27        }
28    }
29
30    int id(ll value) const
31    {
32        int p = lower_bound(vals.begin(), vals.end(), value) - vals.begin();
33        assert(p < (int)vals.size() && vals[p] == value);
34        return p;
35    }
36
37    int new_node()
38    {
```

```

39     if (!free_head)
40     {
41         t.push_back(Node{});
42         return (int)t.size() - 1;
43     }
44     int p = free_head;
45     free_head = t[p].l;
46     t[p] = Node{};
47     return p;
48 }
49
50 int update(int p, int l, int r, int x, int delta)
51 {
52     if (!p) p = new_node();
53     t[p].sum += delta;
54     if (l < r)
55     {
56         int m = (l + r) / 2;
57         if (x <= m)
58         {
59             int child = update(t[p].l, l, m, x, delta);
60             t[p].l = child;
61         }
62         else
63         {
64             int child = update(t[p].r, m + 1, r, x, delta);
65             t[p].r = child;
66         }
67     }
68     if (!t[p].sum)
69     {
70         t[p].l = free_head;
71         free_head = p;
72         return 0;
73     }
74     return p;
75 }
76
77 void modify(int pos, int x, int delta)
78 {
79     for (int i = pos; i <= n; i += i & -i)
80         bit[i] = update(bit[i], 0, (int)vals.size() - 1, x, delta);
81 }

```

```

83 void set(int pos, ll value)
84 {
85     assert(1 <= pos && pos <= n);
86     int x = id(value);
87     if (rank[pos] == x) return;
88     modify(pos, rank[pos], -1);
89     modify(pos, x, 1);
90     rank[pos] = x;
91 }
92
93 ll kth(int l, int r, int k) const
94 {
95     assert(1 <= l && l <= r && r <= n && 1 <= k && k <= r - l + 1);
96     vector<int> a, b;
97     for (int i = r; i; i -= i & -i) a.push_back(bit[i]);
98     for (int i = l - 1; i; i -= i & -i) b.push_back(bit[i]);
99     int L = 0, R = (int)vals.size() - 1;
100     while (L < R)
101     {
102         int count = 0;
103         for (int p : a) count += t[t[p].l].sum;
104         for (int p : b) count -= t[t[p].l].sum;
105         bool left = k <= count;
106         if (!left) k -= count;
107         for (int &p : a) p = left ? t[p].l : t[p].r;
108         for (int &p : b) p = left ? t[p].l : t[p].r;
109         int m = (L + R) / 2;
110         if (left)
111             R = m;
112         else
113             L = m + 1;
114     }
115     return vals[L];
116 }
117 };

```

模板题使用：Luogu P2617

题意：支持单点赋值及闭区间第 k 小。先读完操作，把所有修改的新值收集到 `future`，再构造并按原顺序执行；初值 `vector` 为 0-based，位置、查询区间和 k 为 1-based。Query 仅保存输入字段。

所需模板：DynamicKth。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P2617>

```
1 int main()
2 {
3     struct Query
4     {
5         char op;
6         int x, y, k;
7         long long value;
8     };
9
10    std::ios::sync_with_stdio(false);
11    std::cin.tie(nullptr);
12    int n, m;
13    std::cin >> n >> m;
14    std::vector<long long> a(n), future;
15    for (auto &x : a) std::cin >> x;
16    std::vector<Query> queries(m);
17    for (auto &q : queries)
18    {
19        std::cin >> q.op;
20        if (q.op == 'Q')
21            std::cin >> q.x >> q.y >> q.k;
22        else
23        {
24            std::cin >> q.x >> q.value;
25            future.push_back(q.value);
26        }
27    }
28    DynamicKth tree(a, future);
29    for (const auto &q : queries)
30    {
31        if (q.op == 'Q')
32            std::cout << tree.kth(q.x, q.y, q.k) << '\n';
33        else
34            tree.set(q.x, q.value);
35    }
36 }
```

第九章 K-D Tree

9.1 动态二维 K-D Tree：单点累加与矩形和

KDTreeSum<Coord=int>(reserve_points=0) 初始为空；Coord 取 int 或 long long。add(x,y,delta) 累加有符号 long long 权值，相同坐标只占一个节点；零权节点仍保留，不支持删除坐标。query(x1,y1,x2,y2) 查询闭矩形和，左下端超过右上端则返回 0。坐标只比较，不相减，不用有限哨兵；可用所选整数类型极值。当前各点绝对权值总和及更新中间量须在 long long 内。不同点数 d、预留容量和节点编号须小于 INT_MAX。按轴交替分割，子树过重时递归收集编号并原位重建；树高 $O(\log(d+1))$ 。nth_element 按平均线性选择计，插入摊还 $O(\log^2(d+1))$ ，单次大重建可 $O(d \log(d+1))$ ；矩形查询最坏 $O(d)$ ，不能保证对数。空间 $O(d)$ ，预留容量另计，默认 int 节点常见为 56 字节；紧内存题预留最大不同点数，避免扩容峰值。a/root 及带节点编号的函数均为内部状态；可重新赋值空对象重置。

```
1 template <class Coord = int> struct KDTreeSum
2 {
3     using ll = long long;
4     using Point = array<Coord, 2>;
5
6     struct Node
7     {
8         Point point{}, low{}, high{};
9         array<int, 2> child{};
10        int size = 1;
11        ll value = 0, sum = 0;
12    };
13
14    vector<Node> a;
15    int root = 0;
16
17    KDTreeSum(int reserve_points = 0)
18    {
19        a.reserve(reserve_points + 1);
20        a.push_back(Node{});
21        a[0].size = 0;
22    }
23
24    bool less(Point x, Point y, int axis) const
25    {
26        if (x[axis] != y[axis]) return x[axis] < y[axis];
27        return x[axis ^ 1] < y[axis ^ 1];
28    }
```

包围盒维护、递归收集与重建（接上页同一结构体）：

```

30 void pull(int p)
31 {
32     a[p].low = a[p].high = a[p].point;
33     a[p].size = 1;
34     a[p].sum = a[p].value;
35     for (int q : a[p].child)
36     {
37         if (!q) continue;
38         a[p].size += a[q].size;
39         a[p].sum += a[q].sum;
40         for (int d = 0; d < 2; d++)
41         {
42             a[p].low[d] = min(a[p].low[d], a[q].low[d]);
43             a[p].high[d] = max(a[p].high[d], a[q].high[d]);
44         }
45     }
46 }
47
48 void collect(int p, vector<int> &v)
49 {
50     if (!p) return;
51     collect(a[p].child[0], v);
52     v.push_back(p);
53     collect(a[p].child[1], v);
54 }
55
56 int build(vector<int> &v, int l, int r, int axis)
57 {
58     if (l == r) return 0;
59     int m = l + (r - l) / 2;
60     nth_element(v.begin() + l,
61                 v.begin() + m,
62                 v.begin() + r,
63                 [&](int x, int y) { return less(a[x].point, a[y].point, axis); });
64     int p = v[m];
65     a[p].child[0] = build(v, l, m, axis ^ 1);
66     a[p].child[1] = build(v, m + 1, r, axis ^ 1);
67     pull(p);
68     return p;
69 }

```

点权累加与闭矩形查询（接上页同一结构体）：

```

71     int add(int p, Point point, ll delta, int axis)
72     {
73         if (!p)
74         {
75             p = a.size();
76             a.push_back(Node{});
77             a[p].point = a[p].low = a[p].high = point;
78             a[p].value = a[p].sum = delta;
79             return p;
80         }
81         if (a[p].point == point)
82             a[p].value += delta;
83         else
84         {
85             int side = !less(point, a[p].point, axis);
86             a[p].child[side] = add(a[p].child[side], point, delta, axis ^ 1);
87         }
88         pull(p);
89         int heavy = max(a[a[p].child[0]].size, a[a[p].child[1]].size);
90         if (4LL * heavy <= 3LL * a[p].size) return p;
91         vector<int> v;
92         v.reserve(a[p].size);
93         collect(p, v);
94         return build(v, 0, v.size(), axis);
95     }
96
97     void add(Coord x, Coord y, ll delta) { root = add(root, {x, y}, delta, 0); }
98
99     ll query(int p, Point low, Point high) const
100    {
101        if (!p) return 0;
102        bool covered = true, inside = true;
103        for (int d = 0; d < 2; d++)
104        {
105            if (a[p].high[d] < low[d] || high[d] < a[p].low[d]) return 0;
106            covered &= low[d] <= a[p].low[d] && a[p].high[d] <= high[d];
107            inside &= low[d] <= a[p].point[d] && a[p].point[d] <= high[d];
108        }
109        if (covered) return a[p].sum;
110        ll answer = inside ? a[p].value : 0;
111        for (int q : a[p].child) answer += query(q, low, high);
112        return answer;
113    }
114
115    ll query(Coord x1, Coord y1, Coord x2, Coord y2) const
116    {
117        if (x1 > x2 || y1 > y2) return 0;
118        return query(root, {x1, y1}, {x2, y2});
119    }
120 };

```

应用题补充：Luogu P4148 简单题

题意：在线单点累加与闭矩形求和，每条命令除操作码外都异或上次答案，3 结束。答案保证在 `int` 范围内，因此 `last` 使用 `int`。预留 200000 个不同点，减少 `vector` 扩容峰值；本地验证不等于通过该题 20 MB 平台内存限制。

所需模板：KDTreeSum。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P4148>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, op, last = 0;
6     cin >> n;
7     KDTreeSum<> s(200000);
8     while (cin >> op && op != 3)
9     {
10         int x, y;
11         cin >> x >> y;
12         x ^= last;
13         y ^= last;
14         if (op == 1)
15         {
16             int value;
17             cin >> value;
18             value ^= last;
19             s.add(x, y, value);
20         }
21         else
22         {
23             int x2, y2;
24             cin >> x2 >> y2;
25             x2 ^= last;
26             y2 ^= last;
27             last = s.query(x, y, x2, y2);
28             cout << last << '\n';
29         }
30     }
31 }
```

第十章 动态树

10.1 Link Cut Tree

10.1.1 Link-cut tree: 动态森林与路径异或

构造 LinkCutTree(n), 点号 1..n, 初始权值为零。set(x,value) 修改点权; link(x,y) 仅在不同连通块之间连边并返回是否成功; cut(x,y) 只删除真实存在的边, 否则返回 false。connected 查询连通性; query(x,y) 返回路径点权异或, 要求两点连通, 支持 x=y。make_root/find_root 维护与查询当前表示树根。每次操作均摊 $O(\log n)$, $O(n)$ 空间, splay 下传使用显式栈, 支持长链。权值为 unsigned long long。仅维护路径异或, 不包含虚子树聚合或动态直径。

```
1 struct LinkCutTree
2 {
3     using U = unsigned long long;
4
5     struct Node
6     {
7         int ch[2] {}, fa = 0;
8         U val = 0, sum = 0;
9         bool rev = false;
10    };
11
12    vector<Node> a;
13    vector<int> st;
14
15    LinkCutTree(int n) : a(n + 1), st(n + 1) {}
16
17    bool is_root(int x) const
18    {
19        int f = a[x].fa;
20        return a[f].ch[0] != x && a[f].ch[1] != x;
21    }
22
23    void pull(int x) { a[x].sum = a[a[x].ch[0]].sum ^ a[x].val ^ a[a[x].ch[1]].sum; }
24
25    void reverse(int x)
26    {
27        if (!x) return;
28        swap(a[x].ch[0], a[x].ch[1]);
29        a[x].rev = !a[x].rev;
30    }
31
32    void push(int x)
33    {
34        if (!a[x].rev) return;
35        reverse(a[x].ch[0]);
36        reverse(a[x].ch[1]);
37        a[x].rev = false;
38    }
```

```
40 void rotate(int x)
41 {
42     int y = a[x].fa, z = a[y].fa;
43     int k = a[y].ch[1] == x;
44     int w = a[x].ch[k ^ 1];
45     if (!is_root(y)) a[z].ch[a[z].ch[1] == y] = x;
46     a[x].fa = z;
47     a[y].ch[k] = w;
48     if (w) a[w].fa = y;
49     a[x].ch[k ^ 1] = y;
50     a[y].fa = x;
51     pull(y);
52     pull(x);
53 }
54
55 void splay(int x)
56 {
57     int top = 0, y = x;
58     st[top++] = y;
59     while (!is_root(y))
60     {
61         y = a[y].fa;
62         st[top++] = y;
63     }
64     while (top) push(st[--top]);
65     while (!is_root(x))
66     {
67         int y = a[x].fa, z = a[y].fa;
68         if (!is_root(y))
69         {
70             bool same = (a[y].ch[1] == x) == (a[z].ch[1] == y);
71             rotate(same ? y : x);
72         }
73         rotate(x);
74     }
75 }
```

```
77 void access(int x)
78 {
79     int last = 0;
80     for (int y = x; y; y = a[y].fa)
81     {
82         splay(y);
83         // The old right child's fa remains y: it becomes a path-parent.
84         a[y].ch[1] = last;
85         if (last) a[last].fa = y;
86         pull(y);
87         last = y;
88     }
89     splay(x);
90 }
91
92 void make_root(int x)
93 {
94     access(x);
95     reverse(x);
96 }
97
98 int find_root(int x)
99 {
100     access(x);
101     push(x);
102     while (a[x].ch[0])
103     {
104         x = a[x].ch[0];
105         push(x);
106     }
107     splay(x);
108     return x;
109 }
```

```

111     bool connected(int x, int y) { return x == y || find_root(x) == find_root(y); }
112
113     bool link(int x, int y)
114     {
115         make_root(x);
116         if (find_root(y) == x) return false;
117         a[x].fa = y;
118         return true;
119     }
120
121     bool cut(int x, int y)
122     {
123         make_root(x);
124         access(y);
125         if (a[y].ch[0] != x || a[x].ch[1]) return false;
126         a[y].ch[0] = 0;
127         a[x].fa = 0;
128         pull(y);
129         return true;
130     }
131
132     void set(int x, U value)
133     {
134         access(x);
135         a[x].val = value;
136         pull(x);
137     }
138
139     // x and y must be connected. This is a path aggregate, not a subtree sum.
140     U query(int x, int y)
141     {
142         make_root(x);
143         access(y);
144         return a[y].sum;
145     }
146 };

```

模板题使用：Luogu P3690

题意：维护点权森林，支持路径异或、连边、删边和单点赋值。初始没有边；0 查询、1 连边、2 删边、3 赋值。题目保证查询两点连通，成环的连边和不存在的删边由接口拒绝。点号 1..n。

所需模板：LinkCutTree。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P3690>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m;
6     cin >> n >> m;
7     LinkCutTree g(n);
8     for (int i = 1; i <= n; i++)
9     {
10         unsigned long long x;
11         cin >> x;
12         g.set(i, x);
13     }
14     while (m--)
15     {
16         int op, x, y;
17         cin >> op >> x >> y;
18         if (op == 0)
19             cout << g.query(x, y) << '\n';
20         else if (op == 1)
21             g.link(x, y);
22         else if (op == 2)
23             g.cut(x, y);
24         else
25             g.set(x, y);
26     }
27 }
```

10.1.2 动态换根路径乘积总和

`TreePathProducts<mod>(g,weight)` 要求 `g` 是非空无向树，点号 `0..n-1`、每条边双向存储，`weight` 为等长 `vector<ModInt<mod>>`。`set(u,w)` 修改点权；`query(u)` 返回从 `u` 到所有顶点 `v` 的唯一路径上点权乘积之和，含 `u=v` 的单点路径。固定拓扑，不提供 `link/cut`。建树 $O(n)$ ，修改和查询摊 $O(\log n)$ ，空间 $O(n)$ ，递归建树和 `splay` 下传最坏深度 $O(n)$ 。允许零和负权，`mod` > 1 且不要求素数，不用除法或逆元。`a` 与全部辅助接口仅供内部使用，内部点号 `1..n`；不能把公开接口的 0-based 点号直接传给 `make_root`。依赖 `ModInt`；附件爱莲说的输入 `x` 由主程序显式转为权值 `1-x`。

```

1  template <int mod> struct TreePathProducts
2  {
3      using Z = ModInt<mod>;
4
5      struct Node
6      {
7          int ch[2]{}, fa = 0;
8          Z weight = 0, product = 1, left = 0, right = 0, virt = 0;
9          bool rev = false;
10     };
11
12     vector<Node> a;
13
14     // g is a nonempty tree, with 0-based vertices and one weight per vertex.
15     TreePathProducts(const vector<vector<int>> &g, const vector<Z> &weight)
16         : a(g.size() + 1)
17     {
18         assert(!g.empty() && g.size() == weight.size());
19         auto dfs = [&](auto &&self, int u, int parent) -> void
20         {
21             int x = u + 1;
22             a[x].fa = parent + 1;
23             a[x].weight = weight[u];
24             for (int v : g[u])
25                 if (v != parent)
26                 {
27                     self(self, v, u);
28                     a[x].virt += a[v + 1].left;
29                 }
30             pull(x);
31         };
32         dfs(dfs, 0, -1);
33     }

```

聚合、翻转、递归下传与旋转（接上页同一结构体）：

```

35     bool is_root(int x) const
36     {
37         int f = a[x].fa;
38         return a[f].ch[0] != x && a[f].ch[1] != x;
39     }
40
41     void pull(int x)
42     {
43         auto &t = a[x];
44         const auto &l = a[t.ch[0]], &r = a[t.ch[1]];
45         t.product = l.product * t.weight * r.product;
46         t.left = l.left + l.product * t.weight * (Z(1) + t.virt + r.left);
47         t.right = r.right + r.product * t.weight * (Z(1) + t.virt + l.right);
48     }
49
50     void reverse(int x)
51     {
52         if (!x) return;
53         swap(a[x].ch[0], a[x].ch[1]);
54         swap(a[x].left, a[x].right);
55         a[x].rev = !a[x].rev;
56     }
57
58     void push(int x)
59     {
60         if (!a[x].rev) return;
61         reverse(a[x].ch[0]);
62         reverse(a[x].ch[1]);
63         a[x].rev = false;
64     }
65
66     void push_path(int x)
67     {
68         if (!is_root(x)) push_path(a[x].fa);
69         push(x);
70     }
71
72     void rotate(int x)
73     {
74         int y = a[x].fa, z = a[y].fa;
75         int k = a[y].ch[1] == x, w = a[x].ch[k ^ 1];
76         if (!is_root(y)) a[z].ch[a[z].ch[1] == y] = x;
77         a[x].fa = z;
78         a[y].ch[k] = w;
79         if (w) a[w].fa = y;
80         a[x].ch[k ^ 1] = y;
81         a[y].fa = x;
82         pull(y);
83         pull(x);
84     }

```

Splay、虚实边切换与公开接口（接上页同一结构体）：

```

86     void splay(int x)
87     {
88         push_path(x);
89         while (!is_root(x))
90         {
91             int y = a[x].fa, z = a[y].fa;
92             if (!is_root(y))
93             {
94                 bool same = (a[y].ch[1] == x) == (a[z].ch[1] == y);
95                 rotate(same ? y : x);
96             }
97             rotate(x);
98         }
99     }
100
101     void make_root(int x)
102     {
103         int last = 0;
104         for (int y = x; y; y = a[y].fa)
105         {
106             splay(y);
107             a[y].virt += a[a[y].ch[1]].left - a[last].left;
108             a[y].ch[1] = last;
109             if (last) a[last].fa = y;
110             pull(y);
111             last = y;
112         }
113         splay(x);
114         reverse(x);
115     }
116
117     void set(int u, Z weight)
118     {
119         int x = u + 1;
120         make_root(x);
121         a[x].weight = weight;
122         pull(x);
123     }
124
125     Z query(int u)
126     {
127         int x = u + 1;
128         make_root(x);
129         return a[x].left;
130     }
131 };

```

应用题补充：附件爱莲说接口（本地应用示例）

题意：沿用附件输入： n, q ， n 个 x ， $n-1$ 条 1-based 无向边；1 $u\ x$ 修改参数，2 u 查询。主程序把每个 x 转为点权 $1-x$ ，并把顶点转为 0-based。输出所有从 u 出发路径的点权乘积总和。原题外部地址和线上结果尚未核验。

所需模板：ModInt、TreePathProducts。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://github.com/gsh20040816/CPCtemplate/issues/11>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     using Z = ModInt<998244353>;
6     int n, q;
7     cin >> n >> q;
8     vector<Z> weight(n);
9     for (auto &w : weight)
10    {
11        int x;
12        cin >> x;
13        w = Z(1) - Z(x);
14    }
15    vector<vector<int>> g(n);
16    for (int i = 1; i < n; i++)
17    {
18        int u, v;
19        cin >> u >> v;
20        u--;
21        v--;
22        g[u].push_back(v);
23        g[v].push_back(u);
24    }
25    TreePathProducts<998244353> tree(g, weight);
26    while (q--)
27    {
28        int op, u;
29        cin >> op >> u;
30        if (op == 1)
31        {
32            int x;
33            cin >> x;
34            tree.set(u - 1, Z(1) - Z(x));
35        }
36        else
37            cout << tree.query(u - 1).v << '\n';
38    }
39 }
```

第十一章 配套知识与验证范围

11.1 可持久化区间拼接的懒标记

代码见第 69 页。对象从全零版本开始，同一对象内版本共享坐标域。区间加仅复制被访问节点，不向旧子节点下传标记。节点的和满足 $\text{sum} = \text{left.sum} + \text{right.sum} + \text{lazy} \cdot \text{len}$ 。读取时把祖先累积增量作为参数传下去，因此查询完全只读。

拼接含义为将目标版本某个半开区间替换成源版本同一坐标区间。由于两棵树祖先的 lazy 可能不同，不能直接将源子树挂到目标祖先下面。递归参数 offset 保持“源祖先增量减目标祖先增量”；下降时更新为 $\text{offset} + \text{source.lazy} - \text{destination.lazy}$ 。当区间完整覆盖，复制源节点并将这个 offset 加到其 lazy 和 sum，此后挂到目标祖先下面时，逻辑值才与源版本相同。offset 为零时直接共享源节点；没有覆盖时保留目标节点。空区间、零增量与全区间拼接都可仅增加版本别名。

每一层至多有两个边界节点，其他分支直接返回或复制一个整段节点，所以单次修改、拼接均为 $O(\log n)$ 新节点，查询为 $O(\log n)$ 时间、零新节点。不支持不同坐标偏移的平移粘贴，也不回收仍被旧版本引用的节点。所有 long long 的和、差、懒标记与长度乘积都应在范围内；节点数与版本数须小于 INT_MAX。

验证用独立向量保存各分支版本，逐区间检查更新、拼接和历史版本；包含正负增量、不同祖先标记、空区间、全段共享，以及查询前后的节点数量检查。规模验证包含长度 200000 的 10000 次区间版本组合，并在长度 INT_MAX 的隐式坐标域验证端点操作与共享根。普通与 ASan/UBSan 均通过；区间加 1、后缀拼接和单点读取的应用已获 QOJ8240 双 AC，其余通用接口范围仍由本地验证覆盖。来源需求是 2023 杭州站题解 第 24-26 页的 K 题，完整驱动及验证见下节。

11.2 杭州 2023 K：跨版本拼接应用

[Card Game](#) 的完整驱动在 `verify/qoj/8240/*.cpp`，使用前节的可持久化区间模板。将下标转为从 0 开始，设 $F_i(r)$ 为区间 $[i, r]$ 的剩余牌数， p 为 i 之后第一张与 a_i 相同的牌，若不存在则 $p = n$ 。当 $r < p$ 时，第一张牌始终保留，剩余牌的过程与从 $i + 1$ 开始相同；当 $r \geq p$ 时，到达 p 会将此前区间全部清空。因此

$$F_i(r) = \begin{cases} 1 + F_{i+1}(r), & r < p, \\ F_{p+1}(r), & r \geq p. \end{cases}$$

每个版本的坐标是右端点 r ，值是 $F_i(r)$ 。从 $i = n - 1$ 向前处理：复制版本 $i + 1$ 并在 $[i, p)$ 加 1，再把 $[p, n)$ 拼接自版本 $p + 1$ 。 $p = n$ 时省去后一步。询问原问题的 $[l, r]$ 时，从版本 $l - 1$ 读取坐标 $r - 1$ 的单点值。端点先与上次答案异或解码，不能将输入查询预先离线排序。

$n, q \leq 300000$ ，牌号位于 $[1, n]$ 。构建及查询时间分别为 $O(n \log n)$ 、 $O(q \log n)$ 。驱动另计算各次操作可能访问的相交节点数作为分配上界，一次 reserve 足够容纳整个构建过程，避免复制扩容的临时峰值。该计数仅依据区间结构，不影响牌局答案，且同样为 $O(n \log n)$ 。

本地参照按题意逐张放牌，遇重复就实际删除栈中相应后缀。穷举长度不超过 7 的二元牌序列与全部区间，并打乱询问顺序、真实编码异或输入。另有随机牌局、 $n = q = 300000$ 的全异、全同、周期序列闭式检查，以及稀疏重复大数组的短区间逐张模拟。两套驱动均在线 AC，各 41 个正式测试和 3 个额外测试全过：[动态版](#) 最慢 789 ms、452688 KB，[传统版](#) 最慢 752 ms、452492 KB。此应用仅调用区间加 1、后缀拼接、单点读取；一般有符号增量与区间求和仍由前节本地测试支持。

11.3 并查集模板题的接口范围

P3367 输入点号为 1-based，驱动统一减一后调用 `dsu`。记录 [297668102](#) 通过全部 20 个测试，最慢单点 196 ms、峰值内存 2.27 MB。该输出验证 `merge`、`same` 及其调用的 `find`，不验证 `size`、`groups`；后两者已由独立标签参考和 50 万点长链测试核对。

11.4 动态树路径乘积：虚子树与双向信息

附件“爱莲说”对应的量是

$$F(u) = \sum_{v \in V} \prod_{x \in \text{path}(u, v)} w_x.$$

包括单点路径，所以一个孤立点的答案是其点权，而不是 1。固定根后，递推为 $F(u) = w_u(1 + \sum_{v \text{ 为子节点}} F(v))$ 。使用第 91 页的 `TreePathProducts`，可以单点修改并动态选取查询根。它维护固定树，不包含连边、删边。

辅助树按实链顺序维护三项：整条链点权积 P 、从链首出发的贡献 L 、从链尾出发的贡献 R 。每个点另记虚儿子贡献之和 G 。对于左、右辅助子树，合并式为

$$P = P_l w P_r, \quad L = L_l + P_l w(1 + G + L_r), \quad R = R_r + P_r w(1 + G + R_l).$$

空辅助树取 $P = 1, L = R = 0$ 。翻转时必须同时交换左右儿子和 L, R ；只翻儿子会破坏换根后的方向。

`access` 把旧右链变为虚儿子，将它的 L 加入 G ；把新右链变为实儿子，从 G 减去它的 L 。旧右链的父指针保留为路径父亲。修改点权前先将该点换为根，确保旧贡献已经沿访问路径正确移出虚子树项；然后修改权值并更新合并信息。换根结束时根处的 L 就是所求总和。整个过程只用加减乘，没有除法，零权、负权与合数模数同样适用。

接口使用 0-based 顶点；内部辅助树为 1-based，内部函数不可直接作为外部 API。附件输入参数 x 对应 $w = 1 - x$ ，这个转换在使用程序中显式完成。从源码可确定上述递推和输入协议，但原题外部地址尚未核验，因此只登记本地应用用法。建树 $O(n)$ ，修改和查询摊还 $O(\log n)$ ，总空间 $O(n)$ ；建树和辅助树下传保持递归，最坏深度 $O(n)$ 。本地以逐终点路径枚举、模 8、零权、20 万点链与星形及反复换根验证，不将普通 LCT 的线上记录视为这个变体的 AC。

11.5 隐式 Treap 位翻转与顺序反转

附件 FHQ Treap 中名为 `Reverse` 的操作是逐位异或 1；它没有交换左右儿子，不是区间顺序反转。

`flip_bits(l, r)` 明确表达前者，要求区间元素均为 0 或 1；原有 `reverse(l, r)` 表达后者。两者均使用 1-based 闭区间。例如 0011 的前三位取反得到 1101，而前三位顺序反转得到 1001。二进制序列的区间和就是 1 的个数，仍可直接调用 `query`。

取反可写作 $x \mapsto 1 - x$ 。用符号标记与加法标记表示 $x \mapsto sx + b$ ，其中 $s \in \{1, -1\}$ 。取负时同时令 $s \leftarrow -s, b \leftarrow -b$ ，再加 1；下传时必须先符号、再加法。若把先后顺序颠倒，已有加法会变成错误的符号。顺序反转则只交换儿子并翻转其标记，它与逐元素的取负、加法可交换。结构体公开位翻转接口仅用于二进制值；所有标记和取负中间量仍须在 `long long` 范围内，不能对 `LLONG_MIN` 直接取负。

附件 `Build` 按中点建立树后给节点随机优先级，未修复优先级堆序；这不能直接继承随机 Treap 的平衡复杂度证明。本库逐次 `insert` 构建，期望 $O(n \log n)$ ，每次分裂合并均维持堆序；不能把这种构建成本写为线性。随机优先级下单次操作期望 $O(\log n)$ ，不保证针对已知种子的自适应对抗操作的最坏界。分裂、合并及中序输出保持递归。附件缺少完整牛客题号和主程序，因此新增使用例只作为本地接口示例。

11.6 替罪羊树：重复计数与墓碑清理

第 61 页使用三个不同的量：`size` 计元素副本总数，`nodes` 计物理节点数，`live` 计仍有副本的不同键数。重复值增加计数，删除最后一个副本后保留墓碑。查询排名只累计副本数，不能用物理节点数代替。

更新回溯时，若某孩子占节点数的比例超过 $3/4$ ，或者有效键不足物理节点的一半，就递归中序收集活节点，按中点重建。重建复用活节点编号，保留重复次数。重建后的中序和计数均不变，故集合与排名不变；所有新子树均满足平衡与有效比例。令 d 为当前不同键数，树高为 $O(\log(d+2))$ ，因为每层节点数缩小到至多 $3/4$ ，根的物理节点数至多为 $2d$ 。查询最坏对数，更新摊还对数；一次重建仍可能线性。删除清理后的墓碑不回收到节点池，空间按累计分配量计算，不按当前集合大小计算。

附件的 Rank 只有命中相等键才加一，未命中时可能少一；仅插入 1 后查询 2 的排名就是一个反例。本版统一定义为严格较小副本数加一，空树中的排名也为 1。严格前驱后继使用 `optional`，避免数值哨兵与合法键冲突；完整 `long long` 键无需做减一或加一。P3369 保证第 k 小及邻居存在，使用程序才能直接解引用结果。所有递归保持原样；本地排序向量对拍、重建不变量及 20 万键规模均已验证，线上待补。

11.7 线段树维护单调栈：双向扫描与接雨水

`MonotoneStackSeg` 维护区间加后的运行最大值贡献。每段的 F_0, F_1 分别为从左、从右扫描的最高值减原值之和。若外部高度 x 不低于整段最大值，贡献为 $x \cdot \text{len} - \text{sum}$ 。否则，将孩子按扫描顺序记为 A, B ：若 $x \geq \max A$ ，递归 B ；否则递归 A ，而 B 的贡献已经是 $F_{\text{parent}} - F_A$ 。每次只递归一个孩子，因此单段带边界扫描为 $O(\log n)$ 。区间平移不改变这些差值，懒标记只改最大值、原值和与标记本身。区间修改回溯及任意子区间扫描最坏 $O(\log^2(n+1))$ ，建树 $O(n)$ 。

对全局前缀和后缀最大值 L_i, R_i ，有 $\max(L_i, R_i) = M$ ，其中 M 为全局最大值。因此

$$\sum_i (\min(L_i, R_i) - a_i) = F_0 + F_1 - (nM - \sum_i a_i).$$

根上即可 $O(1)$ 返回雨水体积，不必查询最大点位置。整体平移不改变体积，负高度也成立；带外部高度的 `scan` 则明确使用调用者的边界。所有乘法、和、标记及中间运算须在 `long long` 内，不能只检查最终答案。附件原题为 NERC 2023 G，使用例属于比赛应用，不计正式模板题覆盖；小规模有符号穷举、三十万次随机更新和二十万点规模本地验证通过，线上待补。

11.8 动态 K-D Tree：矩形和与重建边界

`KDTreeSum` 按深度交替坐标轴，以另一轴打破平局。重复坐标累加权值，其他点沿轴全序递归插入；孩子过重时收集编号并用 `nth_element` 重建，复用原节点。包围盒从本点初始化，仅合并非空孩子，不用有限正负无穷；重建直接返回根编号，不用坐标 -1 作为状态标志。权值为零时仍保留节点，负权不影响几何剪枝。

每个孩子大小至多为父节点的 $3/4$ ，所以树高为 $O(\log(d+1))$ ；矩形部分相交却可能递归两边，查询最坏仍为 $O(d)$ 。按平均线性选择计，一次大小 s 的重建需 $O(s \log(s+1))$ ，插入摊还 $O(\log^2(d+1))$ ；不是每次操作的最坏对数界。矩形使用闭边界，只比较坐标、不做坐标减法；各点绝对权值和及更新中间量须在 `long long` 内。

P4148 的在线输入需逐参数 XOR 上次答案，驱动预留二十万个节点以避免扩容峰值。本地默认 `int` 坐标节点为 56 字节，另有重建编号和运行环境开销；该分配量不能代替线上 20 MB 内存通过记录。独立 `map` 对拍、分割与包围盒审计、坐标极值及二十万规模已本地验证，线上待补。

第一部分

跨类依赖

附录 A 数论

A.1 模算术简介

A.1.1 静态模整数

模板参数必须为不超过 `INT_MAX` 的素数且 ≥ 2 。加减乘、`pow`、`inv`、除法；求逆要求非零，`pow` 指数非负。复杂度：加减乘 $O(1)$ ，幂和逆 $O(\log \text{mod})$ 。支持 `+=`、`-=`、`*=`、`/=`，返回自身引用，允许自赋值和链式运算；可作为 Fenwick 区间和的值类型。复合除法沿用非零元素、素数模数前提。

```
1 template <int mod> struct ModInt
2 {
3     int v;
4
5     ModInt(long long x = 0) : v((x % mod + mod) % mod) {}
6
7     ModInt operator+(ModInt b) const { return ModInt((long long)v + b.v); }
8
9     ModInt operator-(ModInt b) const { return ModInt((long long)v - b.v); }
10
11    ModInt operator*(ModInt b) const { return ModInt(1LL * v * b.v); }
12
13    ModInt pow(long long e) const
14    {
15        assert(e >= 0);
16        ModInt a = *this, r = 1;
17        for (; e >>= 1, a = a * a)
18            if (e & 1) r = r * a;
19        return r;
20    }
21
22    ModInt inv() const
23    {
24        assert(v);
25        return pow(mod - 2);
26    } // prime modulus
27
28    ModInt operator/(ModInt b) const { return *this * b.inv(); }
29
30    ModInt &operator+=(ModInt b) { return *this = *this + b; }
31
32    ModInt &operator-=(ModInt b) { return *this = *this - b; }
33
34    ModInt &operator*=(ModInt b) { return *this = *this * b; }
35
36    ModInt &operator/=(ModInt b) { return *this = *this / b; }
37 };
```

算法索引

AffineSegTree, 23
cc_hash_table, 1
dsu, 4
DynamicKth, 78
Fenwick, 11
GcdSequenceTreap, 53
gp_hash_table, 1
gp_map, 1
KDTreeSum, 82
lazy_segtree, 12
LazySeg, 22
LCT
 虚子树, 97
LeftistHeap, 9
LiChao, 37
LinkCutTree, 86
ModInt, 100
MonotoneStackSeg, 26
OrderedSplay, 56
OrderedTreap, 44
ost, 42
PersistentArray, 67
PersistentDistinct, 75
PersistentKth, 65
PersistentRange, 69
pheap, 8
RollbackDSU, 6
rp, 40
ScapegoatTree, 61
SegmentLiChao, 31
segtree, 17
SequenceTreap, 48
Treap
 位翻转, 97
TreePathKth, 72
TreePathProducts, 91

动态树 DP, 97

替罪羊树, 97